



# Deep Koopman-operator-based model predictive control for free-floating space robots with disturbance observer

Renhao Mao<sup>a</sup>, Tao Meng<sup>a,b,\*</sup>, Kun Wang<sup>c</sup>, Jiakun Lei<sup>d</sup>, Weijia Wang<sup>a</sup>

<sup>a</sup> School of Aeronautics and Astronautics, Zhejiang University, Hangzhou, 310027, China

<sup>b</sup> Huanjiang Laboratory, Zhuji, 311899, China

<sup>c</sup> Hangzhou Innovation Institute, Beihang University, Hangzhou, 310051, China

<sup>d</sup> Hangzhou International Innovation Institute, Beihang University, Hangzhou, 311115, China

## ARTICLE INFO

Communicated by Tsourdos Antonios

### Keywords:

Free-floating space robot  
Koopman operator  
Deep neural networks  
Disturbance observer  
Model predictive control

## ABSTRACT

This paper investigates the optimal tracking control problem of free-floating space robots in the presence of non-ideal factors, such as model uncertainties and external disturbances. To address this issue, we first leverage the deep Koopman operator, facilitated with a deep neural network, to establish an offline formulation of a global linearization model for a micro-nano free-floating space robot. Based on the estimated linearization model, we then employ the model predictive control method for online optimization control, achieving a significantly reduced computational burden. Additionally, a decay factor is integrated into the model predictive control optimization objective function to balance the precision of joint angles tracking with the suppression of base satellite attitude disturbance. To further address the modeling inaccuracies and external disturbances, we incorporate a disturbance observer based on a radial basis function neural network for online compensation within the model predictive control framework. This augmentation enhances tracking precision and robustness. Several groups of simulation results are carried out to demonstrate the effectiveness of the proposed method, showing its capability of energy consumption, disturbance rejection and enhanced robustness. These highlight the potential of the proposed method to improve the control performance of free-floating space robots, even in the absence of a precise dynamic model.

## 1. Introduction

Due to their ability to enhance spacecraft capability for executing complex on-orbit tasks, space robots designed for on-orbit spacecraft have attracted significant attention [1]. In the field of space robots, micro-nano space robots are gaining recognition for their cost-effectiveness and adaptable deployment capabilities. However, micro-nano space robots, typically weighing in the range of hundreds or even several kilograms, exhibit more complex dynamic coupling characteristics due to the higher inertia and mass ratios of their manipulators and base satellites compared to larger space robots [2]. This makes it more difficult to establish accurate dynamic models, which complicates the control problem.

Extensive research has been conducted in the field of space robot control. To address model uncertainties and external disturbances, researchers have proposed sliding mode control [3–6], fuzzy control [7]

and adaptive control [8], aiming to achieve high-precision and robust control of space robots. To mitigate the chattering effect caused by the switching of the sliding mode surface, an intelligent adaptive method based on sliding mode control was introduced [9]. Considering the need for rapid control speed in space missions, fixed-time and predefined-time control methods have been proposed in [10,11], reducing the convergence time. To guarantee transient and steady-state performance, the prescribed performance control method is adopted in [12]. While these various control methods address different control challenges, they are fundamentally based on specific physical models. To enhance the accuracy of dynamic models, Sands [13] developed deterministic artificial intelligence techniques by integrating system identification, physics-based methodologies, and adaptive control. This approach incorporates the self-awareness statement of deterministic artificial intelligence proposed in [14], using the 2-norm optimal solution to estimate model uncertainties online. Building on deterministic artificial intelligence tech-

\* Corresponding author at: School of Aeronautics and Astronautics, Zhejiang University, Hangzhou, 310027, China.  
E-mail address: [mengtao@zju.edu.cn](mailto:mengtao@zju.edu.cn) (T. Meng).

## Nomenclature

|                                  |                                                   |                                |                                                           |
|----------------------------------|---------------------------------------------------|--------------------------------|-----------------------------------------------------------|
| LQR                              | linear quadratic regulator                        | $\sigma_i$                     | activation functions                                      |
| MPC                              | model predictive control                          | $\bar{y}_i$                    | outputs for each layer transformation                     |
| NMPC                             | nonlinear model predictive control                | $S$                            | multi-step prediction in loss function                    |
| EDMD                             | extending dynamic mode decomposition              | $\mathcal{L}$                  | loss function                                             |
| DNN                              | deep neural network                               | $\mathcal{L}_1, \mathcal{L}_2$ | components of the loss function                           |
| DK                               | deep Koopman operator                             | $\ \theta_{\bar{W}}\ _2^2$     | $l_2$ regularization term                                 |
| DO                               | disturbance observer                              | $\alpha_1, \alpha_2, \alpha_3$ | hyperparameters of loss function                          |
| RBFNN                            | radial basis function neural network              | $\Delta A, \Delta B$           | approximation errors from truncating the nonlinear system |
| RBFNN-DO                         | DO based on RBFNN                                 | $\Delta F(z(k)), \delta(k)$    | components of the uncertainties                           |
| DOF                              | degrees of freedom                                | $d(k)$                         | disturbances in the lifted space                          |
| ReLU                             | rectified linear units                            | $G_i(z(k))$                    | Gaussian function                                         |
| UUB                              | uniformly ultimately bounded                      | $\zeta_i$                      | center of each Gaussian function                          |
| ISS                              | input-to-state stable                             | $h_i$                          | width of each Gaussian function                           |
| DH                               | Denavit-Hartenberg                                | $l$                            | number of Gaussian functions                              |
| RRMSE                            | relative root mean square error                   | $G(z(k))$                      | composed of $G_i(z(k))$                                   |
| DK-MPC                           | MPC control based on DK model                     | $W^*(k)$                       | optimal weights matrix of RBFNN                           |
| DK-LQR                           | LQR control based on the DK model                 | $\varepsilon^*(z(k))$          | approximation error of RBFNN                              |
| EDMD-MPC                         | MPC control based on EDMD model                   | $\hat{W}(k)$                   | estimation of $W^*(k)$                                    |
| MSE                              | mean square error                                 | $\tilde{W}(k)$                 | estimation error of optimal weights                       |
| EV                               | error variance                                    | $D(k)$                         | lumped disturbances                                       |
| SMAE                             | sum of maximum absolute error                     | $\xi(k)$                       | variable of RBFNN-DO                                      |
| ISCI                             | integral of square control input                  | $\hat{z}(k)$                   | estimation of $z(k)$                                      |
| IV                               | input variance                                    | $\eta, \gamma, \alpha$         | hyperparameters of RBFNN-DO                               |
| IBSAD                            | integral base satellite attitude disturbance      | $P(k)$                         | virtual variable of DO                                    |
| $\dot{q}$                        | velocity vector of the base satellite             | $\hat{P}(k)$                   | estimation of $\hat{P}(k)$                                |
| $v_0$                            | linear velocities of the base satellite           | $\hat{D}(k)$                   | estimation of $\hat{D}(k)$                                |
| $\omega_0$                       | angular velocities of the base satellite          | $\Delta \hat{F}(z(k))$         | estimation of $\Delta F(z(k))$                            |
| $\theta$                         | joint angles of the manipulator                   | $X_{ref}$                      | time-varying reference trajectory                         |
| $H_b$                            | inertia matrix of the base satellite              | $\Gamma(k)$                    | decay factor                                              |
| $H_m$                            | inertia matrix of the manipulator                 | $\psi(k)$                      | variable of decay factor                                  |
| $H_{bm}$                         | coupling inertia matrix                           | $\psi_{\min}, \psi_{\max}$     | hyperparameters of decay factor                           |
| $c_b, c_m$                       | vectors of nonlinear terms                        | $L(\cdot)$                     | each stage cost function of MPC                           |
| $u$                              | control torque acting on the manipulator          | $L_f(\cdot)$                   | terminal cost function of MPC                             |
| $x_\theta$                       | Joint angles and angular velocities               | $N_p$                          | prediction horizon of MPC                                 |
| $x$                              | composed of $x_\theta$ and $\omega_0$             | $Q_1, Q_2, Q_N, R$             | weights of MPC                                            |
| $h(\cdot)$                       | dynamical system evolution                        | $u_{\min}^t$                   | lower bound of control inputs for each joint              |
| $\Delta_b$                       | disturbances exerted on the base satellite        | $u_{\max}^t$                   | upper bound of control inputs for each joint              |
| $\Delta_m$                       | disturbances exerted on the manipulator           | $u^t$                          | control input of the $t$ -th joint                        |
| $f(\cdot)$                       | discrete dynamical system evolution               | $w(k)$                         | estimation error of disturbances                          |
| $\mathcal{K}$                    | Koopman operator                                  | $Y$                            | bound of disturbances estimation error                    |
| $g(\cdot)$                       | observable function                               | $X_{ref}^{\theta_i}$           | desired joint angles                                      |
| $g_x(\cdot)$                     | part of the observable function                   | $X_{ref}^{\theta_i}$           | desired joint angular velocities                          |
| $z(k)$                           | approximation of $g_x(\cdot)$                     | $z_E(x)$                       | embedding function of EDMD                                |
| $K$                              | finite-dimensional approximation of $\mathcal{K}$ | $\varphi_{Ei}(x)$              | basis functions of the lifted space in EDMD               |
| $K_{xx}, K_{xu}, K_{ux}, K_{uu}$ | block matrices of $K$                             | $\zeta_i$                      | kernel centers of basis functions in EDMD                 |
| $A, B$                           | $A = K_{xx}$ and $B = K_{xu}$                     | $J_{LQR}$                      | cost function of DK-LQR                                   |
| $\Phi(x(k))$                     | basis of the lifted space                         | $J_{NMPC}$                     | cost function of NMPC                                     |
| $X, U, Y$                        | datasets collected from the dynamics              | $e_i(k)$                       | angles tracking errors of each joint                      |
| $\Theta, \Omega$                 | $X$ and $Y$ in lifted space                       | $e(k)$                         | composed of $e_i(k)$                                      |
| $\bar{l}$                        | number of Residual blocks                         | $\mu_e$                        | average of $e(k)$                                         |
| $\bar{W}_i$                      | weight matrices of the DNN                        | $\mu_u$                        | average of $u(k)$                                         |

niques, methods such as whiplash compensation [15] and curve flattening [16] were employed to improve the control performance of flexible space robots.

For micro-nano space robots, due to limited on-orbit resources and control abilities, the utilization of optimized control strategy offers advantages as it reduces energy consumption. In the context of optimized control methods, the linear quadratic regulator (LQR) provides an analytical solution for the optimal control gain by directly solving the Riccati equation for linear systems, but it is unable to handle constraints [17]. In nonlinear optimized control methods, adaptive dynamic programming approximates the Hamilton-Jacobi-Bellman equation using a neural network to obtain approximate optimal solutions [18]. However, for complex models, achieving approximate optimal solution convergence online remains a significant challenge, potentially leading to system instability. Model predictive control (MPC), a constrained optimal control strategy that computes optimal control inputs by minimizing a specified objective function over a finite prediction horizon, has seen widespread application in aerospace research [19–21]. Although MPC is effective, traditional MPC is designed for linear models. Given that space robots are nonlinear and dynamically coupled systems, applying MPC directly to space robot control is challenging. Based on the MPC concept, some research has utilized nonlinear model predictive control (NMPC) for space robots [22,23]. However, due to its reliance on complex nonlinear models, NMPC requires precise modeling and imposes a significant computational burden, making it difficult to apply in micro-nano space robots.

Recently, the Koopman operator has garnered significant attention for data-driven modeling. Introduced by B.O. Koopman in 1931 [24], the fundamental concept of the Koopman operator involves globally linearizing a nonlinear system within an infinite-dimensional embedding space. The ability of the Koopman operator to linearize systems facilitates simpler controller design and reduces the computational burden, leading to its increased adoption in aerospace research [25–27]. Given the impracticality of accessing an infinite-dimensional space in real applications, it's necessary to use finite-dimensional lifted observable functions for effective system approximation. Initially, researchers utilized the Koopman operator in control system design through the extending dynamic mode decomposition (EDMD) method [28]. EDMD employs radial basis or polynomial functions to approximate nonlinear models in finite dimensions. However, the manual selection of suitable lifted observable functions for complex nonlinear systems has been a significant challenge. In recent years, with the advancement of artificial intelligence, attention has shifted towards training deep neural networks (DNN) to identify lifted observable functions, has replaced traditional manual methods and become the mainstream trend [29–32]. This method collects input-state data offline, uses the Koopman operator to derive a high-dimensional global linearization model, and trains DNN as lifted observable functions (a more detailed description of how the Koopman operator works is provided in Section 3). The Koopman operator has emerged as a novel approach for MPC implementation in nonlinear systems. References [33–35] used the deep Koopman operator (DK) to construct high-dimensional linearized models of nonlinear systems, which they then applied MPC for controller design. This approach significantly reduces optimization time by eliminating the need for a traditional model-based methodology, unlike NMPC, and avoids the complexity of dynamic modeling.

However, it is unavoidable to encounter discrepancies in modeling between the high-dimensional linear model derived from the Koopman operator and the original nonlinear system model. Furthermore, real-world systems inevitably encounter disturbances, which can significantly affect control efficacy. To address both internal and external disturbances, researchers have also conducted undertaken investigations. Zhang et al. [36] designed a tube-based MPC controller based on the Koopman operator, improving the system's control precision and robustness to disturbances. Chen et al. [37] designed an adaptive MPC controller for time-varying parameter systems based on the Koopman

operator, enhancing the robustness of the controlled system after system parameters switched. While these approaches demonstrate promising theoretical outcomes, the complexity and direct applicability of tube-based or adaptive MPC remain challenging [38]. Consequently, certain studies have embraced robust model predictive control strategies for controller design. Zhu et al. [39] employed a discrete nonlinear disturbance observer to address modeling errors and external disturbances in the Koopman operator, offering direct compensation through MPC framework feedback. However, this approach could risk violating input constraints. Ren et al. [40] uses an extended state disturbance observer to estimate the total disturbance, ignoring the differences between modeling errors and external disturbances.

Unlike fixed-base manipulators, free-floating space robots experience disturbances at the base satellites due to coupling effects when maneuvering their joints, particularly affecting micro-nano space robots significantly. The base satellite attitude disturbance caused by joint angles' movements in turn affect the manipulator, impacting the control effectiveness. Thus, examining the control strategy for free-floating space robots with the base satellite attitude disturbance is meaningful [41]. However, in a unified MPC framework, efforts to minimize the base satellite attitude disturbance can unintentionally reduce the accuracy of joint angles tracking.

In summary, developing a constrained optimal control method for micro-nano space robots that reduces the computational burden, accounts for both internal and external disturbances, avoids complex dynamic models, and suppresses base satellite attitude disturbances, presents a challenge. In this paper, we initially utilize DK to establish a global linearization model of micro-nano space robots using DNN, laying the groundwork for subsequent optimization controller design. To handle modeling inaccuracies and external disturbances, we utilize a disturbance observer (DO) based on a radial basis function neural network (RBFNN) within the MPC framework, thereby improving the precision and robustness of the controller without violating input constraints. Additionally, to suppress base satellite attitude disturbance, we introduce an optimization objective function into MPC with a decay factor to suppress the base satellite attitude disturbance without compromising tracking precision.

The main contributions of this paper are as follows:

- (1) The data-driven modeling method based on the DK is applied to a free-floating space robot, achieving a global approximation linearization model of the free-floating space robot solely through the collection of system inputs and outputs data. To the best of our knowledge, few studies have focused on applying DK to free-floating space robots.
- (2) To mitigate modeling errors from the Koopman operator and external disturbances, a DO based on an RBFNN (RBFNN-DO) embedded within MPC is designed. This improves joint angles tracking precision and enhances robustness to external disturbances, all while adhering to input constraints.
- (3) In response to base satellite attitude disturbance caused by maneuvering joint angles, an objective function with a decay factor is proposed. This function seeks to balance the trade-off between minimizing base satellite attitude disturbance and maintaining joint angles tracking precision, thereby reducing the base satellite attitude disturbance without compromising tracking precision and enhancing the dynamic characteristics of the tracking process.

The rest of this paper is organized as follows: The problem is formulated in Section 2. Section 3 introduces the fundamental principles of the Koopman operator, and establishes a DK model for the free-floating space robot. Section 4 details the design of RBFNN-DO embedded within MPC, incorporating a decay factor in the MPC objective function. Section 5 conducts numerical simulations. Section 6 conducts analyses and discussions of the simulation results. Section 7 concludes the paper.

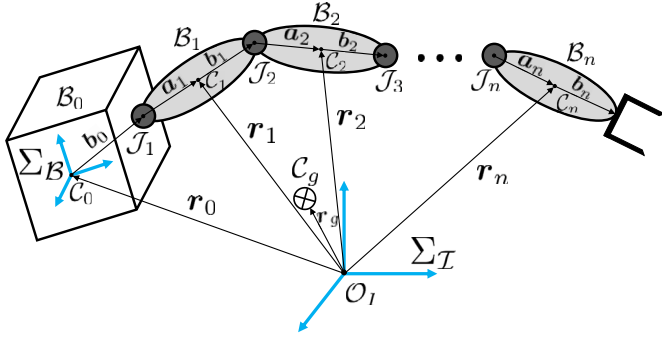


Fig. 1. Model of a free-floating space robot.

**Table 1**  
Free-floating space robot symbols and variables.

| Symbol or Variable                | Definition                                |
|-----------------------------------|-------------------------------------------|
| $B_0$                             | base satellite                            |
| $B_i$                             | Link $i$ , $i = 1, 2, \dots, n$           |
| $J_i$                             | Joint $i$ , $i = 1, 2, \dots, n$          |
| $C_0$                             | center of mass (CM) of the base satellite |
| $C_i$                             | CM of the $B_i$                           |
| $C_g$                             | CM of the system                          |
| $\Sigma_I$                        | the inertial coordinate frame             |
| $\Sigma_B$                        | the base satellite coordinate frame       |
| $r_g \in \mathbb{R}^3$            | position vector of the space robot        |
| $r_0 \in \mathbb{R}^3$            | position vector of the base satellite     |
| $r_i \in \mathbb{R}^3$            | position vector of $B_i$                  |
| $m_0 \in \mathbb{R}^1$            | mass of the base satellite                |
| $m_i \in \mathbb{R}^1$            | mass of $B_i$                             |
| $I_0 \in \mathbb{R}^{3 \times 3}$ | inertia matrix of the base satellite      |
| $I_i \in \mathbb{R}^{3 \times 3}$ | inertia matrix of $B_i$                   |
| $b_0 \in \mathbb{R}^1$            | length from $C_0$ to $J_1$                |
| $a_i \in \mathbb{R}^1$            | length from $J_i$ to $C_i$                |
| $b_i \in \mathbb{R}^1$            | length from $C_i$ to $J_{i+1}$            |

**Notations:** For positive integers  $n$  and  $m$ ,  $\mathbf{0}_n \in \mathbb{R}^n$  denotes a vector with all value 0.  $\mathbf{1}_n \in \mathbb{R}^n$  denotes a vector with all value 1.  $\mathbf{I}_n \in \mathbb{R}^{n \times n}$  denotes the  $n \times n$  identity matrix. For arbitrary vector  $\mathbf{a} \in \mathbb{R}^n$ , matrix  $\mathbf{A} \in \mathbb{R}^{m \times n}$  and positive-definite matrix  $\mathbf{W} \in \mathbb{R}^{n \times n}$ ,  $\mathbf{a}^\top$  denotes the transpose of  $\mathbf{a}$ ,  $\mathbf{a}^\dagger$  denotes the Moore-Penrose pseudoinverse of  $\mathbf{a}$ ,  $\|\mathbf{a}\|_2 = \sqrt{\mathbf{a}^\top \mathbf{a}}$ ,  $\|\mathbf{a}\|_2^2 = \mathbf{a}^\top \mathbf{a}$ ,  $\|\mathbf{a}\|_W^2 = \mathbf{a}^\top \mathbf{W} \mathbf{a}$ ,  $\|\mathbf{A}\|_F$  denotes the Frobenius norm of  $\mathbf{A}$ ,  $\mathbf{a}(k+i|k)$  denotes the value of  $\mathbf{a}$  at time  $k+i$ , predicted at time  $k$ , if  $i=0$ ,  $\mathbf{a}(k|k) = \mathbf{a}(k)$ .

## 2. Problem formulation

Consider a space robot consisting of a base satellite and a manipulator with  $n$  degrees of freedom (DOF), as shown in Fig. 1. The symbols and variables of the space robot are defined in Table 1.

The dynamic equation expressed as follows is excited to collect data for training in this paper [42]:

$$\begin{bmatrix} \mathbf{H}_b & \mathbf{H}_{bm} \\ \mathbf{H}_{bm}^\top & \mathbf{H}_m \end{bmatrix} \begin{bmatrix} \ddot{\mathbf{q}} \\ \ddot{\boldsymbol{\theta}} \end{bmatrix} + \begin{bmatrix} \mathbf{c}_b \\ \mathbf{c}_m \end{bmatrix} = \begin{bmatrix} \mathbf{0}_6 \\ \mathbf{u} \end{bmatrix} \quad (1)$$

where  $\dot{\mathbf{q}} = [\mathbf{v}_0^\top \ \boldsymbol{\omega}_0^\top]^\top \in \mathbb{R}^6$  is the velocity vector of the base satellite including the linear velocities  $\mathbf{v}_0 \in \mathbb{R}^3$  and the angular velocities  $\boldsymbol{\omega}_0 \in \mathbb{R}^3$ .  $\boldsymbol{\theta} = [\theta_1 \ \theta_2 \ \dots \ \theta_n]^\top \in \mathbb{R}^n$  is the joint angles of the manipulator, and  $\theta_1, \theta_2, \dots, \theta_n$  denote the joint angle of each joint.  $\mathbf{H}_b \in \mathbb{R}^{6 \times 6}$  and  $\mathbf{H}_m \in \mathbb{R}^{n \times n}$  are inertia matrices of the base satellite and the manipulator.  $\mathbf{H}_{bm} \in \mathbb{R}^{6 \times n}$  is the coupling inertia matrix between the base satellite and the manipulator.  $\mathbf{c}_b \in \mathbb{R}^6$  and  $\mathbf{c}_m \in \mathbb{R}^n$  are vectors of nonlinear terms.  $\mathbf{u} \in \mathbb{R}^n$  is the control torque acting on the manipulator.

Space robot is a multi-input and multi-output nonlinear system with its output being strong coupled which makes the controller design dif-

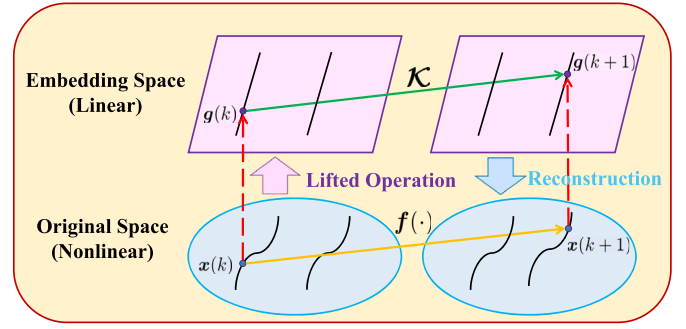


Fig. 2. Schematic of the Koopman operator operating on nonlinear system.

ficult. What is worse, in the real-world the dynamics (1) is usually unknown, especially for micro-nano space robots [43].

**Assumption 1.** The joint angles, joint angular velocities and the base satellite angular velocities of the free-floating space robot are all measurable.

Considering the tracking control of the manipulator's joint angles and the suppression of base satellite attitude disturbance during the tracking control process. Therefore, refer to Assumption 1, a dataset containing joint angles, joint angular velocities and base satellite angular velocities can be obtained. Define  $\mathbf{x}_\theta = [\theta_1 \ \dots \ \theta_n \ \dot{\theta}_1 \ \dots \ \dot{\theta}_n]^\top \in \mathbb{R}^{2n}$ ,  $\mathbf{x} = [\mathbf{x}_\theta^\top \ \boldsymbol{\omega}_0^\top]^\top \in \mathbb{R}^{2n+3}$ . The dynamic Equation (1) can be succinctly represented as follows:

$$\dot{\mathbf{x}} = \mathbf{h}(\mathbf{x}, \mathbf{u}) \quad (2)$$

where  $\mathbf{h} : \mathbb{R}^{2n+3} \times \mathbb{R}^n \rightarrow \mathbb{R}^{2n+3}$  is a smooth vector field of  $C^\infty$ , describing the evolution of the dynamical system.

In practical scenarios, free-floating space robots inevitably encounter external disturbances. Consequently, Equation (1) can be reformulated as follows:

$$\begin{bmatrix} \mathbf{H}_b & \mathbf{H}_{bm} \\ \mathbf{H}_{bm}^\top & \mathbf{H}_m \end{bmatrix} \begin{bmatrix} \ddot{\mathbf{q}} \\ \ddot{\boldsymbol{\theta}} \end{bmatrix} + \begin{bmatrix} \mathbf{c}_b \\ \mathbf{c}_m \end{bmatrix} = \begin{bmatrix} \mathbf{0}_6 \\ \mathbf{u} \end{bmatrix} + \begin{bmatrix} \Delta_b \\ \Delta_m \end{bmatrix} \quad (3)$$

where  $\Delta_b \in \mathbb{R}^6$  and  $\Delta_m \in \mathbb{R}^n$  denote the disturbances exerted on the base satellite and the manipulator, respectively.

**Assumption 2.** The disturbances  $\Delta_b$  and  $\Delta_m$  affecting the free-floating space robot, and their derivatives, are bounded.

## 3. DK for modeling free-floating space robots

This section begins with an introduction to the fundamental principles of the Koopman operator. Subsequently, we establish a high-dimensional approximate linearization model for a free-floating space robot based on the Koopman operator using DNN, laying the groundwork for controller design.

### 3.1. Koopman operator

The Koopman operator is widely used to describe uncontrolled nonlinear dynamical systems [44,45]. Through a series of straightforward transformations, it can also be adapted to characterize dynamical systems that incorporate control inputs. The operational procedure of the Koopman operator on the nonlinear system is illustrated in Fig. 2.

The dynamical system of the free-floating space robot (2) in discrete form can be presented as:

$$\mathbf{x}(k+1) = \mathbf{f}(\mathbf{x}(k), \mathbf{u}(k)) \quad (4)$$

where  $\mathbf{x}(k) \in \mathbb{R}^{2n+3}$  and  $\mathbf{u}(k) \in \mathbb{R}^n$  denote the states and control inputs of the dynamical system at time  $k$ , respectively.  $\mathbf{f} : \mathbb{R}^{2n+3} \times \mathbb{R}^n \rightarrow \mathbb{R}^{2n+3}$  is a smooth vector field of  $C^\infty$ , describing the evolution of the discrete dynamical system.

Koopman operator  $\mathcal{K}$  is an infinite-dimensional linear transformation acting on the observable function  $\mathbf{g}$ , and can be described as:

$$\mathbf{g}(\mathbf{x}(k+1), \mathbf{u}(k+1)) = \mathcal{K}\mathbf{g}(\mathbf{x}(k), \mathbf{u}(k)) \quad (5)$$

where  $\mathbf{g} : \mathbb{R}^{2n+3} \times \mathbb{R}^n \rightarrow \mathbb{R}^N$  is a smooth vector field of  $C^\infty$ , mapping the state space into the designated embedding space. The embedding function  $\mathbf{g}$  can be further separated into two parts in terms of states and control inputs:

$$\mathbf{g}(\mathbf{x}(k), \mathbf{u}(k)) = \begin{bmatrix} \mathbf{g}_x(\mathbf{x}(k)) \\ \mathbf{u}(k) \end{bmatrix} \quad (6)$$

where  $\mathbf{g}_x : \mathbb{R}^{2n+3} \rightarrow \mathbb{R}^{N-n}$ .

The infinite-dimensional  $\mathcal{K}$  is impossible to realize in applications, but can be truncated with a finite-dimensional approximation [28]. Equation (5) can be simplified within finite dimension space [33]:

$$\begin{bmatrix} \mathbf{z}(k+1) \\ \mathbf{u}(k+1) \end{bmatrix} = \mathbf{K} \begin{bmatrix} \mathbf{z}(k) \\ \mathbf{u}(k) \end{bmatrix} = \begin{bmatrix} \mathbf{K}_{xx} & \mathbf{K}_{xu} \\ \mathbf{K}_{ux} & \mathbf{K}_{uu} \end{bmatrix} \begin{bmatrix} \mathbf{z}(k) \\ \mathbf{u}(k) \end{bmatrix} \quad (7)$$

where  $\mathbf{z}(k) \in \mathbb{R}^{n_z}$  is the approximation of embedding function  $\mathbf{g}_x$ .  $\mathbf{K} \in \mathbb{R}^{(n_z+n) \times (n_z+n)}$  is the finite-dimensional approximation of  $\mathcal{K}$ .  $\mathbf{K}_{xx} \in \mathbb{R}^{n_z \times n_z}$ ,  $\mathbf{K}_{xu} \in \mathbb{R}^{n_z \times n}$ ,  $\mathbf{K}_{ux} \in \mathbb{R}^{n \times n_z}$  and  $\mathbf{K}_{uu} \in \mathbb{R}^{n \times n}$  are block matrices of  $\mathbf{K}$ .

Following the definition provided in Equation (7), let  $\mathbf{K}_{xx} = \mathbf{A}$  and  $\mathbf{K}_{xu} = \mathbf{B}$ . The evolution of the dynamical system within the lifted space is described as follows:

$$\mathbf{z}(k+1) = \mathbf{A}\mathbf{z}(k) + \mathbf{B}\mathbf{u}(k) \quad (8)$$

Compared to Equation (7), Equation (8) offers enhanced feasibility for subsequent controller design.

To reconstruct the original state space from the lifted space at time  $k$ ,  $\mathbf{z}(k)$  is defined as follows:

$$\mathbf{z}(k) = \begin{bmatrix} \mathbf{x}(k) \\ \Phi(\mathbf{x}(k)) \end{bmatrix} \quad (9)$$

where  $\Phi(\mathbf{x}(k)) = [\varphi_1(\mathbf{x}(k)) \ \varphi_2(\mathbf{x}(k)) \ \cdots \ \varphi_{n_z-2n-3}(\mathbf{x}(k))]^\top \in \mathbb{R}^{n_z-2n-3}$  is the basis of the lifted space. Consequently, the state space is encapsulated within the lifted space, allowing for its reconstruction without loss of accuracy through projection onto  $\mathbf{z}(k)$ .

Therefore, for the dynamics Equation (2), a globally linearized model in the lifted space is established as follows:

$$\begin{cases} \mathbf{z}(k+1) = \mathbf{A}\mathbf{z}(k) + \mathbf{B}\mathbf{u}(k) \\ \mathbf{x}_\theta(k) = \mathbf{C}_1\mathbf{z}(k) \\ \mathbf{u}_0(k) = \mathbf{C}_2\mathbf{z}(k) \end{cases} \quad (10)$$

where  $\mathbf{C}_1 = [\mathbf{1}_{2n}^\top \ \mathbf{0}_{n_z-2n}^\top]^\top$ ,  $\mathbf{C}_2 = [\mathbf{0}_{2n}^\top \ \mathbf{1}_3^\top \ \mathbf{0}_{n_z-2n-3}^\top]^\top$ .

Let  $\mathbf{X} \in \mathbb{R}^{(2n+3) \times (M-1)}$ ,  $\mathbf{U} \in \mathbb{R}^{n \times (M-1)}$ , and  $\mathbf{Y} \in \mathbb{R}^{(2n+3) \times (M-1)}$  represent the datasets collected from the dynamics Equation (2), respectively:

$$\begin{aligned} \mathbf{X} &= [\mathbf{x}(1) \ \mathbf{x}(2) \ \cdots \ \mathbf{x}(M-1)] \\ \mathbf{U} &= [\mathbf{u}(1) \ \mathbf{u}(2) \ \cdots \ \mathbf{u}(M-1)] \\ \mathbf{Y} &= [\mathbf{x}(2) \ \mathbf{x}(3) \ \cdots \ \mathbf{x}(M)] \end{aligned} \quad (11)$$

where  $\mathbf{x}(i)$  and  $\mathbf{u}(i)$  respectively denote the system states at time  $i$  and the corresponding control inputs.

The original system states collected are mapped to the lifted space, represented by the matrices  $\Theta \in \mathbb{R}^{n_z \times (M-1)}$  and  $\Omega \in \mathbb{R}^{n_z \times (M-1)}$ :

$$\begin{aligned} \Theta &= [\mathbf{z}(1) \ \mathbf{z}(2) \ \cdots \ \mathbf{z}(M-1)] \\ \Omega &= [\mathbf{z}(2) \ \mathbf{z}(3) \ \cdots \ \mathbf{z}(M)] \end{aligned} \quad (12)$$

Consequently, the matrices  $\mathbf{A}$  and  $\mathbf{B}$  in Equation (8) can be solved by minimizing the following least squares problem, as detailed in [37]:

$$\min_{\mathbf{A}, \mathbf{B}} \|\Omega - \mathbf{A}\Theta - \mathbf{B}\mathbf{U}\|_F \quad (13)$$

The minimum norm solution of Equation (13) is:

$$[\mathbf{A} \ \mathbf{B}] = \Omega [\Theta \ \mathbf{U}]^\dagger \quad (14)$$

### 3.2. Train of DK

In the early research on Koopman operator, the lifted observable functions depended on manual selection. For complex nonlinear systems, selecting appropriate lifted observable functions for finite-dimensional approximation is highly challenging. DNN offers complex mapping relationships, which provides new insights into the selection of lifted observable functions in Koopman operator [46].

Fig. 3 illustrates the framework of training DK using DNN. This paper employs ResNet [47] for training to derive finite-dimensional lifted observable functions. Residual blocks within the ResNet incorporate skip connections, rendering the training process more stable compared to fully connected layers. Matrices  $\mathbf{A}$  and  $\mathbf{B}$  are concurrently obtained during the training process.

**Assumption 3.** The lifted observable functions are Lipschitz continuous, and the elements  $\{\varphi_i(\mathbf{x}(k))\}_{i=1}^{n_z-2n-3}$  in the lifted observable functions, apart from the original states, are linearly independent.

Before training DNN, by selecting appropriate activation functions such as rectified linear units (ReLU) [48] that are Lipschitz continuous, we can ensure the outputs of DNN are Lipschitz continuous. DNN is initialized with random weights during training, and as a result of the training process, a highly complex mapping relationship emerges between the outputs and inputs, making it highly likely to satisfy the condition of linear independence. Thus Assumption 3 can be satisfied.

**Assumption 4.** The lifted observable functions are constructed such that  $\mathbf{z}(\mathbf{0}_{2n+3}) = \mathbf{0}_{n_z}$ .

For the free-floating space robot (2), there exists a feedback control law such that the system states are asymptotically stable at the origin, denoted as  $\mathbf{x} \rightarrow \mathbf{0}_{2n+3}$ . Similarly, for the high-dimensional system (10), it is hypothesized that a feedback control law exists to guarantee the asymptotic stability of all states at the origin.

The importance of Assumption 4 will be further elucidated in Section 5.2.

By employing activation functions such as the ReLU, centered at zero, and setting biases within the neural network to zero, we adhere to Assumption 4 during the construction of the DNN architecture. Therefore, different from the DNN formulations presented in [49–51], this paper opts to exclude biases in the DNN design. Specifically, the expression for the ResNet adopted in this paper is delineated below:

$$\begin{cases} \bar{\mathbf{y}}_0 = \sigma_0(\bar{\mathbf{W}}_0 \mathbf{x}) \\ \bar{\mathbf{y}}_1 = \sigma_1(\bar{\mathbf{W}}_1(\sigma_1(\bar{\mathbf{W}}_1 \bar{\mathbf{y}}_0) + \bar{\mathbf{y}}_0)) \\ \vdots \\ \bar{\mathbf{y}}_{\bar{l}} = \sigma_{2\bar{l}}(\bar{\mathbf{W}}_{2\bar{l}}(\sigma_{2\bar{l}-1}(\bar{\mathbf{W}}_{2\bar{l}-1} \bar{\mathbf{y}}_{\bar{l}-1}) + \bar{\mathbf{y}}_{\bar{l}-1})) \\ \Phi = \bar{\mathbf{W}}_{2\bar{l}+1} \bar{\mathbf{y}}_{\bar{l}} \end{cases} \quad (15)$$

where  $\bar{l}$  denotes the number of Residual blocks within the ResNet architecture.  $\{\bar{\mathbf{W}}_i\}_{i=0}^{2\bar{l}+1}$  and  $\{\sigma_i\}_{i=0}^{2\bar{l}}$  denote the weights and activation functions associated with the ResNet.  $\{\bar{\mathbf{y}}_i\}_{i=0}^{\bar{l}}$  denotes the outputs corresponding to each transformation within the specified range.

The DNN hyperparameters for the simulations in Section 5 are detailed in Table 3 in Section 5.1.



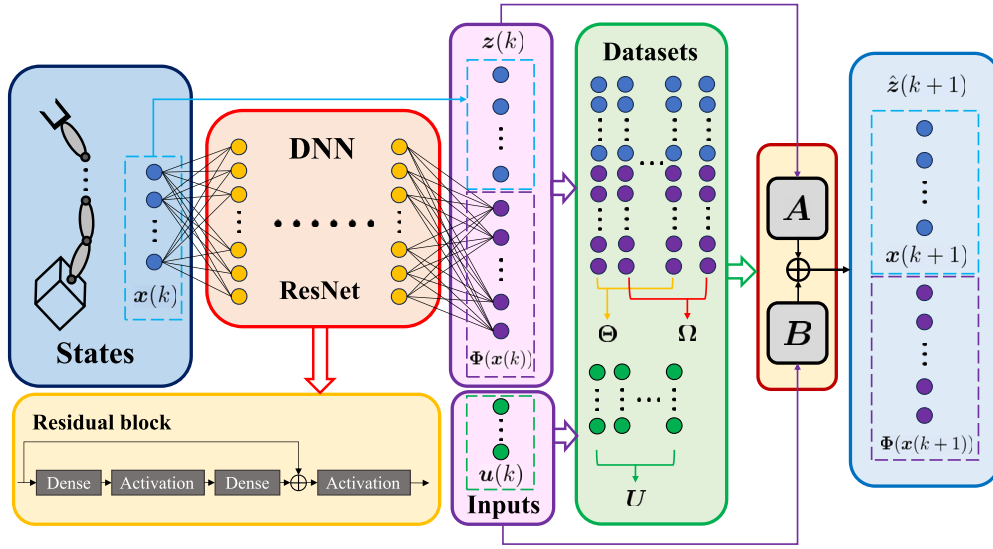


Fig. 3. Framework of training DK using DNN.

To train the DK, it is necessary to collect data using the dynamical model (2), obtaining a series of state trajectories by applying random control inputs. Each state trajectory comprises a sequence of state-input pairs evolving over time. Consequently, in the construction of the loss function during the training process, a multi-step prediction format can be adopted to enhance the training effectiveness.

Assume that the system states at time  $k$  are represented by  $x(k)$ , with control inputs denoted as  $u(k)$ , and the states in the lifted space are given by  $z(k)$ . The prediction of the system states in the lifted space after  $S$  steps can be denoted as:

$$\hat{z}(k+S) = A^S z(k) + \sum_{i=1}^S A^{S-i} B u(k+i-1) \quad (16)$$

The loss function selected in this paper is mainly composed of two parts. The first part is the prediction error function of the system in the lifted space, which refers to the error of the system's prediction  $S$  steps after time  $k$ , and can be defined as:

$$\mathcal{L}_1 = \sum_{i=1}^S \|z(k+i) - \hat{z}(k+i)\|_2^2 \quad (17)$$

The second part is the prediction error of the original system states. Although we have collected states regarding the manipulator's joint angles, joint angular velocities and base satellite angular velocities, the design of the subsequent controller focuses more on the restoration of the manipulator's joint angles and angular velocities. Therefore, this portion of the loss function can be defined as:

$$\mathcal{L}_2 = \sum_{i=1}^S \|x_\theta(k+i) - C_1 \hat{z}(k+i)\|_2^2 \quad (18)$$

The training objective of the DNN is to minimize the following loss function:

$$\mathcal{L} = \alpha_1 \mathcal{L}_1 + \alpha_2 \mathcal{L}_2 + \alpha_3 \|\theta_{\bar{W}}\|_2^2 \quad (19)$$

where  $\alpha_1$ ,  $\alpha_2$  and  $\alpha_3$  are the positive hyperparameters corresponding to different parts of the loss function.  $\|\theta_{\bar{W}}\|_2^2$  represents the  $l_2$  regularization term, used to prevent overfitting.

The gradient descent Adam [52] optimizing algorithm is used for training the DNN.

In the training stage, we usually choose a smaller  $\alpha_3$  than  $\alpha_1$  and  $\alpha_2$ . The hyperparameters for the training process of the DNN (15) in Section 5.2 are detailed in Table 3 in Section 5.1.

Following the training of DNN, a linearized model of the free-floating space robot (3) in the lifted space can be obtained:

$$\begin{aligned} z(k+1) &= (A + \Delta A)z(k) + (B + \Delta B)u(k) + d(k) \\ &= Az(k) + Bu(k) + \Delta F(z(k)) + \delta(k) + d(k) \end{aligned} \quad (20)$$

where  $\Delta A$  and  $\Delta B$  represent approximation errors arising inevitably from the truncation of the original nonlinear system into a finite-dimensional linear space during the training of DNN. A portion of these approximation errors is associated with  $z(k)$ , thus, the uncertainty is depicted in the form of  $\Delta F(z(k))$ , while another segment of the approximation errors is represented by  $\delta(k)$ .  $d(k)$  denotes the expression of the disturbances to the original dynamical model within the lifted space.

**Assumption 5.** In Equation (20), the uncertainties represented by  $\Delta F(z(k))$ ,  $\delta(k)$ ,  $d(k)$ , and their derivatives, are bounded.

The boundedness of external disturbances in the lifted space can be derived from Assumption 2. The boundedness of approximation error from modeling the nonlinear system to the high-dimensional linearized system can be addressed through the design of DNN and the training process. As analyzed in [53], the operation of concatenating the original states of the system with the lifted observable functions produced by DNN yields a high data utilization rate. The DNN component can be regarded as compensating for the residuals in the transformation from a nonlinear system to a linearized system.

#### 4. Controller design

This section utilizes the high-dimensional approximate linearization model (20) obtained from the previous section based on DK to design the controller for the free-floating space robot. Initially, an RBFNN-DO is designed to estimate both internal and external disturbances, and then embeds the estimation of disturbances within the framework of the MPC. Subsequently, a cost function with a decay factor is devised to suppress base attitude disturbance while ensuring precision in tracking the joint angles of the manipulator.

The structure of the proposed method is illustrated in Fig. 4. First, we employ the free-floating space robot (2) to collect data (11). It is essential that the datasets encompass a diverse range. Next, these datasets are employed to train the linearization DK model (10) offline using the DNN (15). In the online control phase, we utilize the RBFNN-DO as described in (23) and (30) to address both internal and external disturbances. These components are integrated into our proposed  $\Gamma$ -MPC

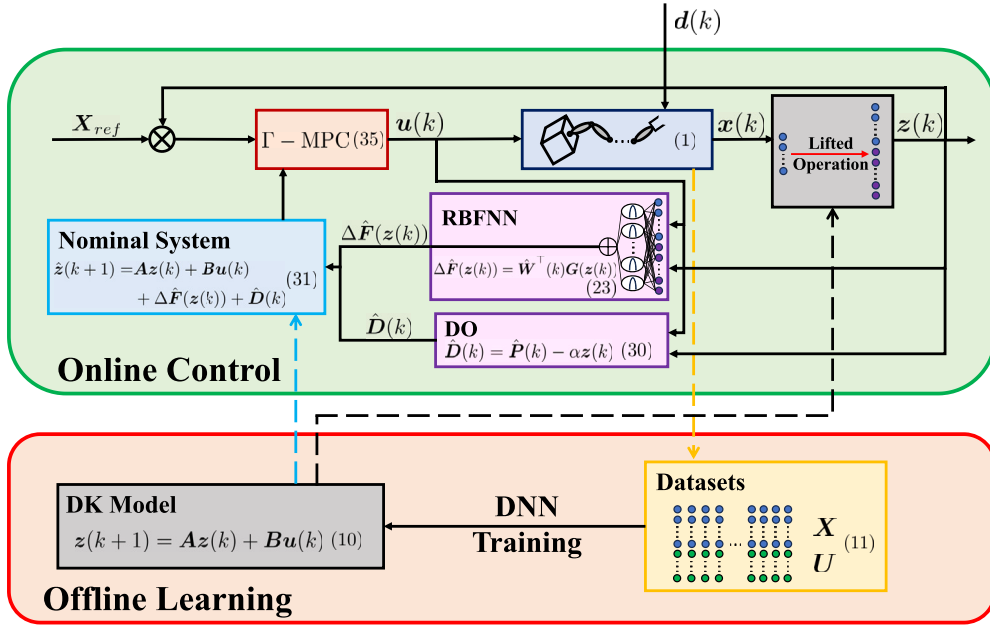


Fig. 4. Structure of the proposed method.

(35) framework. The  $\Gamma$ -MPC leverages the linearized model from the DK transformation and the disturbance compensation from the RBFNN-DO to formulate a constrained optimal control strategy.

#### 4.1. Design of RBFNN-DO

Inspired by the discrete system DO designed in [54], this paper adopts RBFNN to approximate the uncertainty term  $\Delta F(z(k))$  associated with  $z(k)$  in Equation (20). Select the Gaussian function of the following form:

$$G_i(z(k)) = \exp\left(-\frac{\|z(k) - \zeta_i\|_2^2}{2h_i^2}\right) \quad i = 1, 2, \dots, l \quad (21)$$

where  $\zeta_i \in \mathbb{R}^{n_z}$  is the center of each Gaussian function.  $h_i$  is the width of each Gaussian function.  $l$  denotes the number of Gaussian functions used to form the activation Gaussian function  $G(z(k)) \in \mathbb{R}^l$ , and  $G(z(k)) = [G_1(z(k)), G_2(z(k)), \dots, G_l(z(k))]^\top$ .

Thus, the uncertainty  $\Delta F(z(k))$  can be presented as:

$$\Delta F(z(k)) = W^{*\top}(k)G(z(k)) + \varepsilon^*(z(k)) \quad (22)$$

where  $W^*(k) \in \mathbb{R}^{l \times n_z}$  is the optimal weights matrix of RBFNN,  $\varepsilon^*(z(k))$  is the approximation error.

The uncertainty  $\Delta F(z(k))$  can be estimated as:

$$\Delta \hat{F}(z(k)) = \hat{W}^\top(k)G(z(k)) \quad (23)$$

where  $\hat{W}(k)$  is the estimation of  $W^*(k)$ .

The model Equation (20) can be rewritten as:

$$z(k+1) = Az(k) + Bu(k) + \hat{W}^\top(k)G(z(k)) - \tilde{W}^\top(k)G(z(k)) + D(k) \quad (24)$$

where  $D(k) = \varepsilon^*(z(k)) + \delta(k) + d(k)$ ,  $\tilde{W}(k) = \hat{W}(k) - W^*(k)$ .

To apply RBFNN, it is necessary to design an updating law to estimate the weights at the next time step. Unlike the trajectory tracking error-based updating law used in [54], this paper takes into account that the errors resulting from Koopman operator modeling mainly stem from model uncertainty. Therefore, a model error-driven weight updating law is adopted. Define a variable:

$$\xi(k) = z(k) - \hat{z}(k) \quad (25)$$

where  $\hat{z}(k)$  is the estimation of  $z(k)$ .

The updating law is designed as:

$$\hat{W}(k+1) = \hat{W}(k) + \eta G(z(k))\xi^\top(k+1) - \gamma \hat{W}(k) \quad (26)$$

where  $\eta$  is a positive coefficient to be chosen, denoting the learning rate of RBFNN.  $\gamma$  is a small positive coefficient, used to modify the updating law.

To estimate the integrated disturbances in Equation (24), define a virtual variable:

$$P(k) = D(k) - \alpha z(k) \quad (27)$$

where  $\alpha$  is a positive coefficient to be chosen.

The RBFNN-DO hyperparameters for the simulations in Section 5 are detailed in Table 4 in Section 5.1.

According to Equation (24) and Equation (27):

$$P(k+1) = D(k+1) - \alpha (Az(k) + Bu(k) + W^{*\top}(k)G(z(k)) + P(k) + \alpha z(k)) \quad (28)$$

The estimation of  $P(k+1)$  is designed as:

$$\hat{P}(k+1) = -\alpha (Az(k) + Bu(k) + \hat{W}^\top(k)G(z(k)) + \hat{P}(k) + \alpha z(k)) \quad (29)$$

where  $\hat{P}(k)$  is the estimation of  $P(k)$ .

Thus  $D(k)$  can be estimated as:

$$\hat{D}(k) = \hat{P}(k) - \alpha z(k) \quad (30)$$

By integrating Equation (23) and Equation (30) to substitute for the uncertainties in Equation (20), the nominal system for the subsequently controller design is:

$$\hat{z}(k+1) = Az(k) + Bu(k) + \Delta \hat{F}(z(k)) + \hat{D}(k) \quad (31)$$

**Lemma 1.** [55] For system (20), if there exists a positive defined function  $V(x(k))$  with continuous partial differences such that for  $x(k)$  in a compact set  $\Omega_x \subseteq \mathbb{R}^n$ ,  $\Delta V(x(k))$  is negative for  $\|x(k)\|_2 > C$ . And the ball of radius  $C$  is contained in  $\Omega_x$ , the system is uniformly ultimately bounded (UUB) and the norm of the states of the system is bounded within a neighborhood of  $C$ .

**Theorem 1.** Considering the closed-loop system (20), with the neural network employing the weights updating law (26), and the internal and external

disturbances estimated by Equation (23) and Equation (30) respectively, then the estimation error of the disturbances in system (20) is UUB.

**Proof.** The proof of Theorem 1 can be seen in Appendix A.  $\square$

#### 4.2. Design of MPC

The objective of the controller design is to enable the free-floating space robots (3) to track a time-varying reference trajectory  $\mathbf{X}_{ref} \in \mathbb{R}^{2n}$ , with a priority on tracking accuracy, while minimizing the base satellite attitude disturbance.

Different from the approach utilized in [41], which uses a fixed weight to handle the base satellite attitude disturbance while tracking a fixed reference trajectory, this paper advocates for the implementation of a decay factor. This factor engenders a modifiable balance between precision of tracking a time-varying reference trajectory and the reduction of base satellite attitude disturbance. The decay factor's role is to suppress the base satellite attitude disturbance induced by the manipulator when the joint angles converge towards their targets. Concurrently, once the control target for joint angles is achieved, the focus shifts primarily to maintaining tracking precision. This approach helps prevent a potential decline in precision of tracking a time-varying reference trajectory, which might occur when attempting to optimize the base satellite attitude disturbance simultaneously.

To quantify tracking performance, we introduce a variable  $\psi(k) = \|\mathbf{C}_1 \mathbf{z}(k) - \mathbf{X}_{ref}(k)\|_2^2$ . Then, the decay factor is defined as:

$$\Gamma(k) = \begin{cases} 0 & 0 \leq \psi(k) \leq \psi_{\min} \\ \tanh\left(\frac{\psi(k) - \psi_{\min}}{\psi_{\max} - \psi(k)}\right) & \psi_{\min} < \psi(k) < \psi_{\max} \\ 1 & \psi_{\max} \leq \psi(k) \end{cases} \quad (32)$$

where  $\psi_{\min}$  and  $\psi_{\max}$  are positive coefficients to be designed.

Within the framework of MPC, the cost function for each stage and the terminal cost function of our  $\Gamma$ -MPC are defined as:

$$L(\hat{\mathbf{z}}(k+i|k), \mathbf{u}(k+i|k)) = \|\mathbf{C}_1 \hat{\mathbf{z}}(k+i|k) - \mathbf{X}_{ref}(k+i|k)\|_{\mathbf{Q}_1}^2 + \Gamma(k) \|\mathbf{C}_2 \hat{\mathbf{z}}(k+i|k)\|_{\mathbf{Q}_2}^2 + \|\mathbf{u}(k+i|k)\|_{\mathbf{R}}^2 \quad (33)$$

$$L_f(\hat{\mathbf{z}}(k+N_p|k)) = \|\mathbf{C}_1 \hat{\mathbf{z}}(k+N_p|k) - \mathbf{X}_{ref}(k+N_p|k)\|_{\mathbf{Q}_N}^2 \quad (34)$$

where  $N_p$  denotes the prediction horizon.  $\mathbf{Q}_1 \in \mathbb{R}^{2n \times 2n}$ ,  $\mathbf{Q}_2 \in \mathbb{R}^{3 \times 3}$ ,  $\mathbf{Q}_N \in \mathbb{R}^{2n \times 2n}$ ,  $\mathbf{R} \in \mathbb{R}^{n \times n}$  denote weights respectively.

The  $\Gamma$ -MPC hyperparameters for the simulations in Section 5 are detailed in Table 4 in Section 5.1.

Considering the system (20), integrating the RBFNN-DO (23) and (30), along with the designed state cost function (33) and terminal cost function (34), the optimization problem of  $\Gamma$ -MPC can be described as follows:

$$\begin{aligned} \min_{\mathbf{u}(k)} J(k) &= L_f(\hat{\mathbf{z}}(k+N_p|k)) + \sum_{i=0}^{N_p-1} L(\hat{\mathbf{z}}(k+i|k), \mathbf{u}(k+i|k)) \\ \text{subject to } &\begin{cases} \hat{\mathbf{z}}(k) = \mathbf{z}(k) \\ \hat{\mathbf{z}}(k+i+1|k) = \mathbf{A}\hat{\mathbf{z}}(k+i|k) + \mathbf{B}\mathbf{u}(k+i|k) + \Delta\hat{\mathbf{F}}(\mathbf{z}(k)) + \hat{\mathbf{D}}(k) \\ \mathbf{u}'_{\min} \leq \mathbf{u}'(k+i|k) \leq \mathbf{u}'_{\max} \quad i = 1, 2, \dots, n \end{cases} \end{aligned} \quad (35)$$

where  $\mathbf{u}'_{\min}$  and  $\mathbf{u}'_{\max}$  denote the lower bound and upper bound of control inputs for each joint,  $\mathbf{u}'$  denotes the control input of the  $i$ -th joint.

**Theorem 2.** The norm of the prediction errors within the optimization problem (35) between the true system (20) and the nominal system (31), are bounded.

**Proof.** According to Theorem 1 and Assumption 5, by selecting appropriate parameters, the estimation error  $\mathbf{w}(k) = \Delta\hat{\mathbf{F}}(\mathbf{z}(k)) + \hat{\mathbf{D}}(k)$  is

bounded, and in the finite prediction horizon, for each  $\|\mathbf{w}(k+i|k)\|_2 (i = 0, 1, \dots, N_p)$ , satisfies  $\|\mathbf{w}(k+i|k)\|_2 \leq Y$ . The error between nominal system (31) and true system (20) is bounded in:

$$\begin{aligned} \|\hat{\mathbf{z}}(k+1|k) - \mathbf{z}(k+1)\|_2 &= \|\mathbf{w}\|_2 \leq Y \\ \|\hat{\mathbf{z}}(k+2|k) - \mathbf{z}(k+2)\|_2 &= \mathbf{A}\|\mathbf{w}\|_2 + \|\mathbf{w}(k+1|k)\|_2 \leq (\mathbf{A} + \mathbf{I}_{n_z})Y \\ &\vdots \\ \|\hat{\mathbf{z}}(k+i|k) - \mathbf{z}(k+i)\|_2 &\leq \sum_{j=0}^{i-1} \mathbf{A}^j Y = \frac{\mathbf{A}^i - \mathbf{I}_{n_z}}{\mathbf{A} - \mathbf{I}_{n_z}} Y \end{aligned} \quad (36)$$

Theorem 2 has been proven.  $\square$

Several assumptions related to the optimization problem (35) are stated in the following for the analysis of closed stability [56]:

**Assumption 6.** The stage cost function  $L(\hat{\mathbf{z}}(k+i|k), \mathbf{u}(k+i|k))$  is Lipschitz continuous with a Lipschitz constant  $L_a$ , and there exists a  $\mathcal{K}_{\infty}$  function  $a_1$ , such that  $L(\hat{\mathbf{z}}(k+i|k), \mathbf{u}(k+i|k)) \geq a_1(\|\hat{\mathbf{z}}(k+i|k)\|_2)$ .

**Assumption 7.** The terminal cost function  $L_f(\hat{\mathbf{z}}(k+N_p|k))$  is Lipschitz continuous with a Lipschitz constant  $L_b$ , and there exists a  $\mathcal{K}_{\infty}$  function  $a_2$ , such that  $L_f(\hat{\mathbf{z}}(k+N_p|k)) \geq a_2(\|\hat{\mathbf{z}}(k+N_p|k)\|_2)$ .

**Assumption 8.** There exists a local stabilizing controller  $\boldsymbol{\mu}(k)$  for terminal set such that  $L_f(\hat{\mathbf{z}}(k+1)) - L_f(\hat{\mathbf{z}}(k)) \leq L(\hat{\mathbf{z}}(k), \boldsymbol{\mu}(k))$ .

**Lemma 2.** [57] Given the system  $\mathbf{x}(k+1) = \mathbf{f}(\mathbf{x}(k), \mathbf{w}(k))$ , if there exists a positive definite function  $V(\mathbf{x}(k), \mathbf{w}(k))$ , and  $\mathcal{K}_{\infty}$  function  $a_3, a_4, a_5, a_6$ , such that  $a_3(\|\mathbf{x}(k)\|_2) \leq V(\mathbf{x}(k), \mathbf{w}(k)) \leq a_4(\|\mathbf{x}(k)\|_2)$  and  $V(\mathbf{x}(k+1), \mathbf{w}(k+1)) - V(\mathbf{x}(k), \mathbf{w}(k)) \leq -a_5(\|\mathbf{x}(k)\|_2) + a_6(\|\mathbf{w}(k)\|_2)$ , the system is input-to-state stable (ISS).

**Theorem 3.** Considering the system (20) controlled by the proposed  $\Gamma$ -MPC strategy (35), and the RBFNN-DO (23) and (30) are employed, the system is ISS.

**Proof.** The proof of Theorem 3 can be seen in Appendix B.  $\square$

## 5. Simulations

### 5.1. Simulation settings

In this section, the efficacy of the proposed method is evaluated using a 5-DOF micro-nano free-floating space robot, as depicted in Fig. 5 [58]. The Denavit-Hartenberg (DH) parameters and the kinematic and dynamic parameters of this system are delineated in Table 2. The joint torque limitations for each joint are constrained with  $\mathbf{u}'_{\min} = -5\text{Nm} (i = 1, 2, \dots, 5)$  and  $\mathbf{u}'_{\max} = 5\text{Nm} (i = 1, 2, \dots, 5)$ . The sampling time is chosen as  $\Delta T = 0.01\text{s}$ . The offline learning is carried out in Python 3.8.18 and the online controller is conducted in Matlab R2021b, running on a computer with Intel Core i9-12900H 2.50 GHz and 16.0 GB RAM. The hyperparameters selected for the simulations in this paper are shown in Table 3 and Table 4.

### 5.2. Training and prediction evaluation

In this subsection, an offline DK model for the free-floating space robot is constructed using DNN.

The datasets are generated through several trajectories, each comprising 100 steps (1s), in the MATLAB environment using Equation (2) via the Runge-Kutta method. We collect input and output data of the free-floating space robot in Section 5.1 without considering the underlying structure of the model (1). The datasets should encompass a diverse range of joint torques to form the basis for subsequent DK model



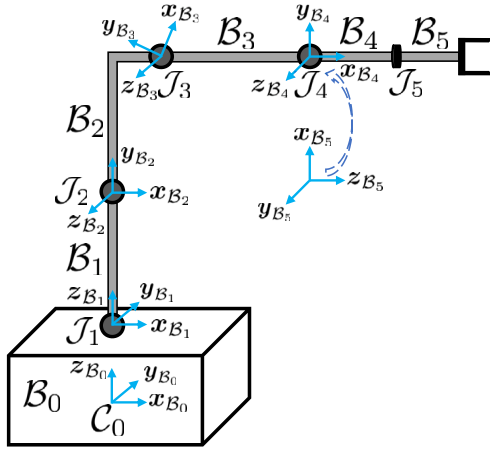


Fig. 5. Simulation model.

Table 2

Parameters of the space robot.

|                                                   | $B_0$ | $J_1$   | $J_2$  | $J_3$   | $J_4$   | $J_5$    |
|---------------------------------------------------|-------|---------|--------|---------|---------|----------|
| Mass (kg)                                         | 50    | 2       | 2      | 2       | 2       | 2        |
| Inertia $I_{xx}$ ( $\text{kg} \cdot \text{m}^2$ ) | 2     | 0.2     | 0.2    | 0.2     | 0.2     | 0.2      |
| Inertia $I_{yy}$ ( $\text{kg} \cdot \text{m}^2$ ) | 1     | 0.1     | 0.1    | 0.1     | 0.1     | 0.1      |
| Inertia $I_{zz}$ ( $\text{kg} \cdot \text{m}^2$ ) | 3     | 0.3     | 0.3    | 0.3     | 0.3     | 0.3      |
| DH. $d$ (m)                                       | -     | 0.125   | 0      | 0       | 0       | 0.15     |
| DH. $\alpha$ (rad)                                | -     | $\pi/2$ | 0      | 0       | $\pi/2$ | $-\pi/2$ |
| DH. $a$ (m)                                       | -     | 0       | 0.1515 | 0.1420  | 0       | 0        |
| DH. $\theta$ (rad)                                | -     | 0       | 1.2553 | -1.2553 | $\pi/2$ | $-\pi/2$ |
| $b^x$ (m)                                         | 0     | 0       | 0.0871 | 0.0710  | 0       | 0.035    |
| $b^y$ (m)                                         | 0     | 0.0625  | 0.0223 | 0       | 0       | 0        |
| $b^z$ (m)                                         | 0.05  | 0       | 0      | 0       | -0.04   | 0        |

Table 3

Hyperparameters of the DK.

| hyperparameter                                               | value                                                   |
|--------------------------------------------------------------|---------------------------------------------------------|
| number of Residual blocks in (15)                            | $\bar{l} = 5$                                           |
| activation functions $\{\sigma_i\}_{i=0}^{2\bar{l}}$ in (15) | ReLU                                                    |
| neurons of each hidden layer in (15)                         | 200                                                     |
| number of observable functions in (7)                        | $n_z = 30$                                              |
| structure of DNN (15)                                        | $[13, 200, \dots, 200, 17]$                             |
| loss function in (16) and (19)                               | $S = 9, \alpha_1 = 1, \alpha_2 = 1, \alpha_3 = 10^{-8}$ |
| learning rate in (15)                                        | $10^{-4}$                                               |

construction. To achieve this, torques are randomly generated within  $[-5, 5]$  Nm for the first two joints and  $[-2, 2]$  Nm for the last three joints, within the control torque range specified in 5.1. We conducted 1010 simulations of the free-floating space robot to generate 1010 trajectories, with 1000 trajectories allocated for training and 10 reserved for testing.

The training loss for the DK model of the free-floating space robot is illustrated in Fig. 6.

Next, we conduct a simple simulation to highlight the importance of Assumption 4. During the DK model training, we perform one set of experiments with setting biases to zero and another set without setting the biases to zero. Subsequently, we validate the simulations using the same control algorithm. All initial joint angles are set to be  $\theta_i(0) = 0^\circ$  ( $i = 1, 2, \dots, 5$ ) and the desired joint angles are set to be  $X_{ref}^{\theta_i}(t) = 0^\circ$  ( $i = 1, 2, \dots, 5$ ). The control results for both cases are shown in Fig. 7.

For comparison, an EDMD [28,36,39,40] is designed to show the effectiveness of DK. The number of observables is chosen to be consistent with DK, set at 30, with 13 representing the original states. In EDMD method, the thin plate spline radial basis function used in [28,36,39,40] is adopted:

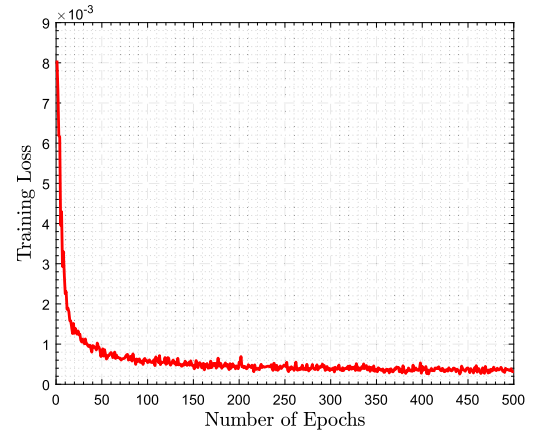


Fig. 6. Training loss of the DK.

$$\mathbf{z}_E(\mathbf{x}) = [\mathbf{x}^\top, \varphi_{E1}(\mathbf{x}) \quad \dots \quad \varphi_{E17}(\mathbf{x})]^\top - [\mathbf{0}_{13}^\top, \varphi_{E1}(\mathbf{0}_{13}) \quad \dots \quad \varphi_{E17}(\mathbf{0}_{13})]^\top \quad (37)$$

$$\varphi_{Ei}(\mathbf{x}) = \|\mathbf{x} - \boldsymbol{\varsigma}_i\|_2^2 / \log(\|\mathbf{x} - \boldsymbol{\varsigma}_i\|_2^2) \quad i = 1, 2, \dots, 17$$

where  $\mathbf{z}_E(\mathbf{x}) \in \mathbb{R}^{30}$  is the embedding function of EDMD,  $\varphi_{Ei}(\mathbf{x})$  ( $i = 1, 2, \dots, 17$ ) represent the basis functions of the lifted space in EDMD and  $\boldsymbol{\varsigma}_i \in \mathbb{R}^{13}$  ( $i = 1, 2, \dots, 17$ ) are kernel centers, each element of  $\boldsymbol{\varsigma}_i$  is chosen randomly in  $[-0.1, 0.1]$ .

The trained models with DK and EDMD are validated with the same training datasets. Fig. 8 illustrates the predictive performance of DK and EDMD.

The relative root mean square error (RRMSE) is used to evaluate the performances of DK and EDMD:

$$\text{RRMSE} = \frac{\sqrt{\sum_{k=1}^{T/\Delta T} \|\mathbf{x}_{pred}(k) - \mathbf{x}_\theta(k)\|_2^2}}{\sqrt{\sum_{k=1}^{T/\Delta T} \|\mathbf{x}_\theta(k)\|_2^2}} \times 100\% \quad (38)$$

where  $\mathbf{x}_{pred}(k)$  denotes the prediction of  $\mathbf{x}_\theta(k)$  at time  $k$ .

The predictive performances of DK and EDMD are illustrated in Table 5.

### 5.3. Tracking precision and robustness evaluation

In this subsection, we will evaluate the advantages of the proposed method. Consequently, we compare the proposed method with the following four approaches: the proposed method without RBFNN-DO (employing MPC based on the DK model, DK-MPC), employing LQR based on the DK model (DK-LQR), employing MPC based on the EDMD model (EDMD-MPC), and traditional NMPC. The proposed method, DK-MPC, and EDMD-MPC are all implemented using the *quadprog* solver [59], while NMPC was optimized by the *fmincon* solver [59], both within the Matlab environment.

For comparison, the cost functions of DK-MPC and EDMD-MPC are the same as that of the proposed method. The cost function of DK-LQR is:

$$J_{LQR} = \|\mathbf{x}_\theta - \mathbf{X}_{ref}\|_{Q_1}^2 + \|\mathbf{u}\|_R^2 \quad (39)$$

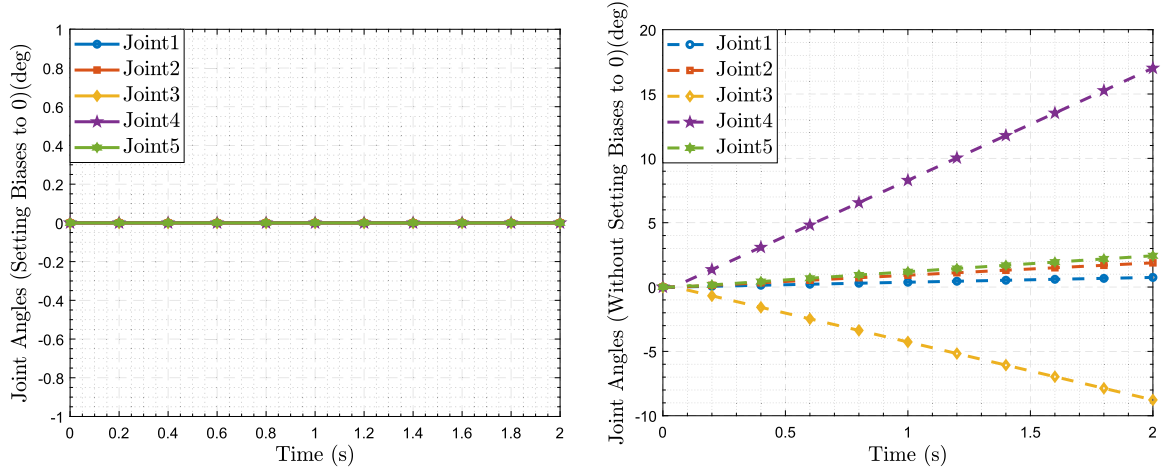


Fig. 7. Control performances of setting biases to 0 and without setting biases to 0.

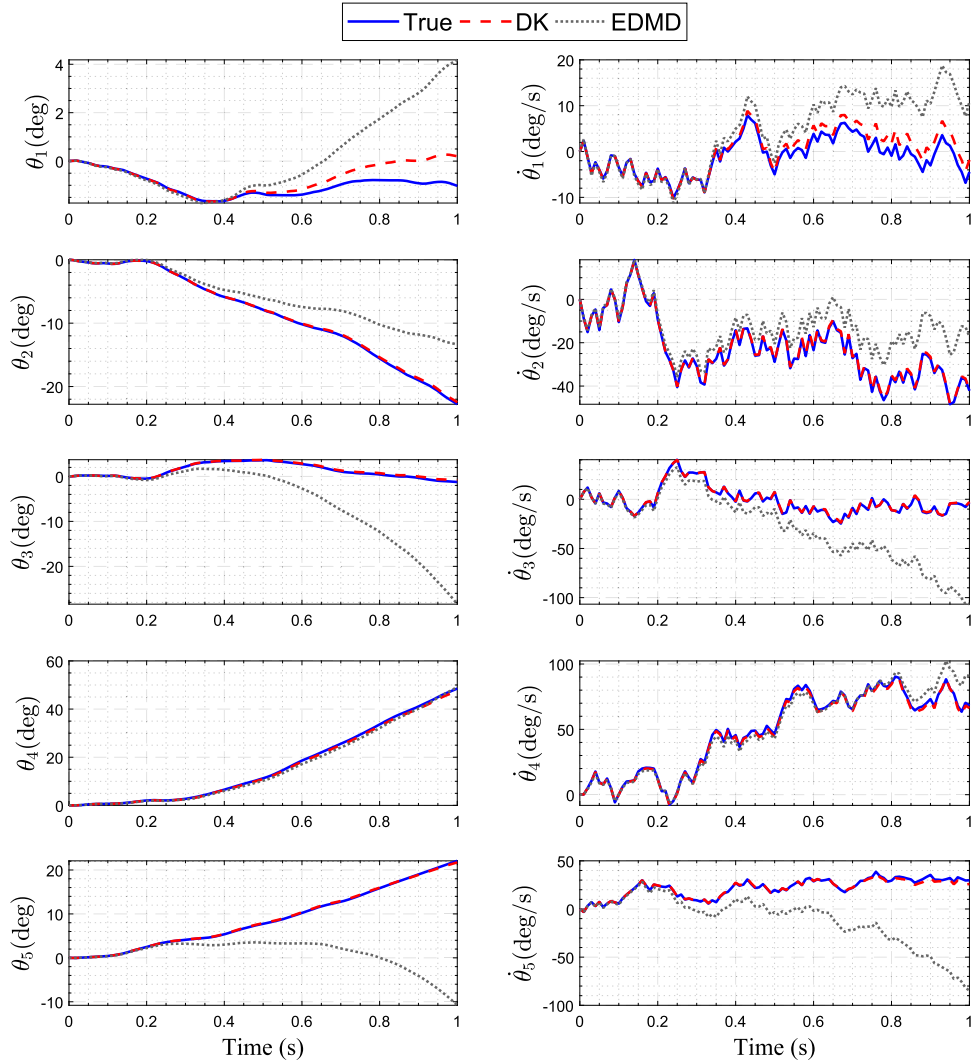
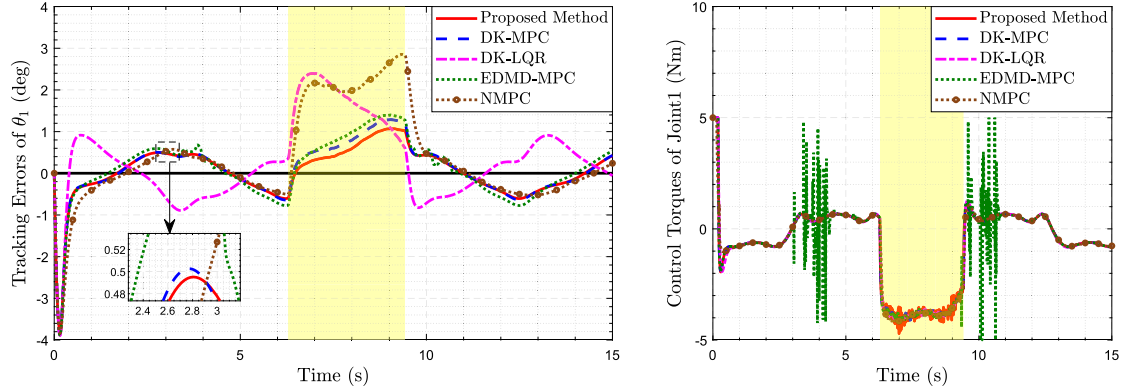


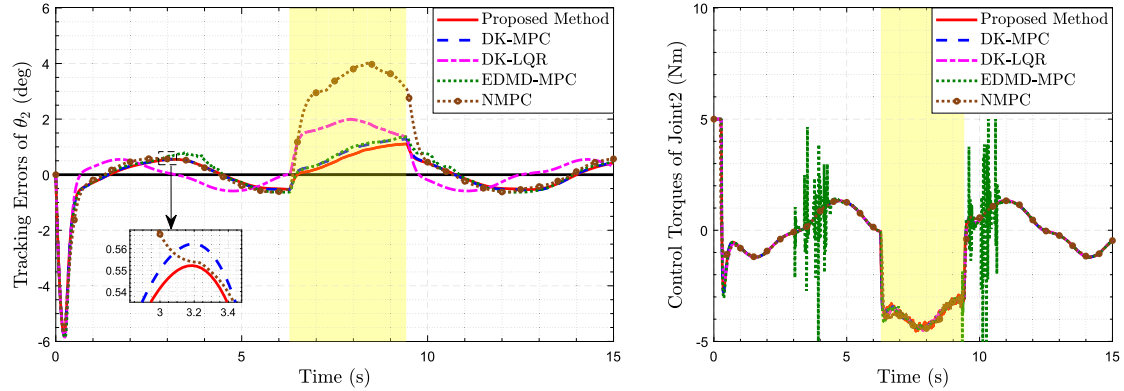
Fig. 8. Predictive results of the free-floating space robot with DK and EDMD.

**Table 4**  
Hyperparameters of the controller.

| hyperparameter | value                                                 |
|----------------|-------------------------------------------------------|
| RBFNN-DO       | number of Gaussian functions in (21)                  |
|                | each element of $\xi_i (i = 1, 2, \dots, 50)$ in (21) |
|                | width of each Gaussian function in (21)               |
|                | initial estimation weights in (23)                    |
|                | RBFNN updating law in (26)                            |
| $\Gamma$ -MPC  | virtual variable in (27)                              |
|                | decay factor (32)                                     |
|                | weights in (33) and (34)                              |
|                | prediction horizon in (34)                            |
|                |                                                       |



**Fig. 9.** Tracking errors and control torques of Joint1. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)



**Fig. 10.** Tracking errors and control torques of Joint2.

**Table 5**  
Predictive performances of DK and EDMD.

| Predict step(s) | 1     | 10     | 50     | 100    |
|-----------------|-------|--------|--------|--------|
| RRMSE (DK)      | 0.69% | 1.17%  | 2.26%  | 3.50%  |
| RRMSE (EDMD)    | 6.22% | 18.91% | 39.62% | 89.63% |

The cost function of NMPC is

$$\begin{aligned}
 \min_{u(k)} J_{NMPC}(k) &= \|x_\theta(k+9|k) - X_{ref}(k+9|k)\|_{Q_N}^2 \\
 &+ \sum_{i=0}^8 \left( \|x_\theta(k+i|k) - X_{ref}(k+i|k)\|_{Q_1}^2 + \|u(k+i|k)\|_R^2 \right) \\
 \text{subject to } &\begin{cases} \begin{bmatrix} H_b & H_{bm} \\ H_{bm}^\top & H_m \end{bmatrix} \begin{bmatrix} \ddot{q} \\ \dot{\theta} \end{bmatrix} + \begin{bmatrix} c_b \\ c_m \end{bmatrix} = \begin{bmatrix} 0_6 \\ u \end{bmatrix} \\ u_{\min}^i \leq u^i(k+i|k) \leq u_{\max}^i \quad i = 1, 2, \dots, 5 \end{cases}
 \end{aligned} \quad (40)$$

Initial joint angles and angular velocities are set to be  $\theta_i(0) = 0^\circ (i = 1, 2, \dots, 5)$ ,  $\dot{\theta}_i(0) = 0^\circ/s (i = 1, 2, \dots, 5)$ . The desired tracking trajectories are  $X_{ref}^{\theta_i}(t) = 50^\circ \cdot \sin(t) (i = 1, 2, \dots, 5)$ ,  $X_{ref}^{\dot{\theta}_i}(t) = 50^\circ/s \cdot \cos(t) (i = 1, 2, \dots, 5)$ , where  $X_{ref}^{\theta_i}(t)$  and  $X_{ref}^{\dot{\theta}_i}(t)$  represent the desired trajectories for each joint and together compose the vector  $X_{ref}$ .

Besides, to evaluate the robustness of the proposed method, unknown disturbances  $\Delta_m^i(t) = (1 + \sin(0.2t) + \cos(0.1t) + e^{-0.1t} + 0.5\text{rand}(1))$  Nm ( $i = 1, 2, \dots, 5$ ) are applied to each joint during  $(2\pi, 3\pi]$ s (yellow zones in Figs. 9 to 13). To maintain fairness, the unknown disturbances fall within the control capabilities of the proposed controller and the controllers for comparison.

The simulation results of the tracking errors and control torques of each joint are shown in Figs. 9 to 13.

In addition to demonstrating the control performances, several indices are used to evaluate the effectiveness of the four controllers: Mean square error (MSE), error variance (EV), sum of maximum absolute error (SMAE), integral of square control input (ISCI) and input variance

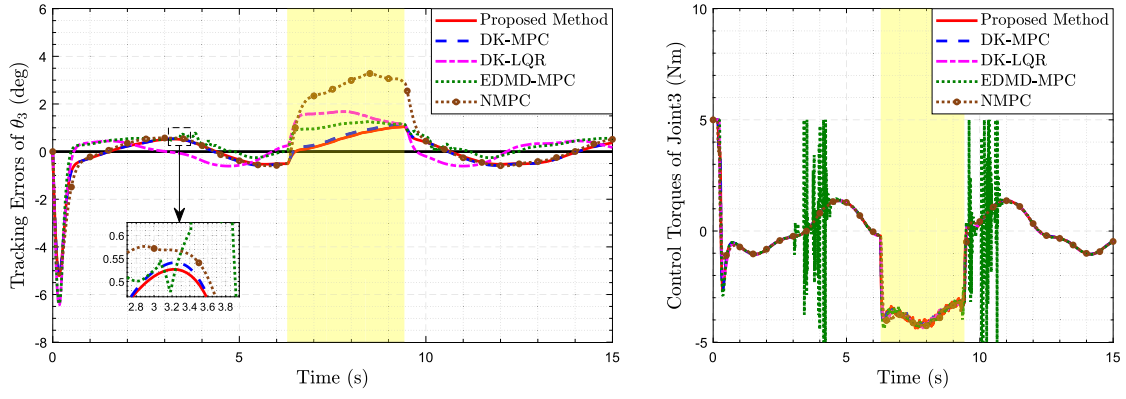


Fig. 11. Tracking errors and control torques of Joint3.

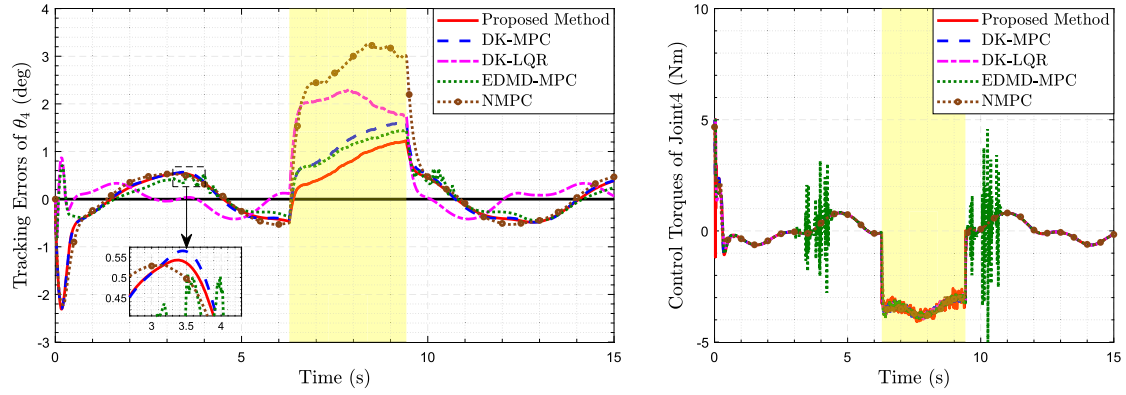


Fig. 12. Tracking errors and control torques of Joint4.

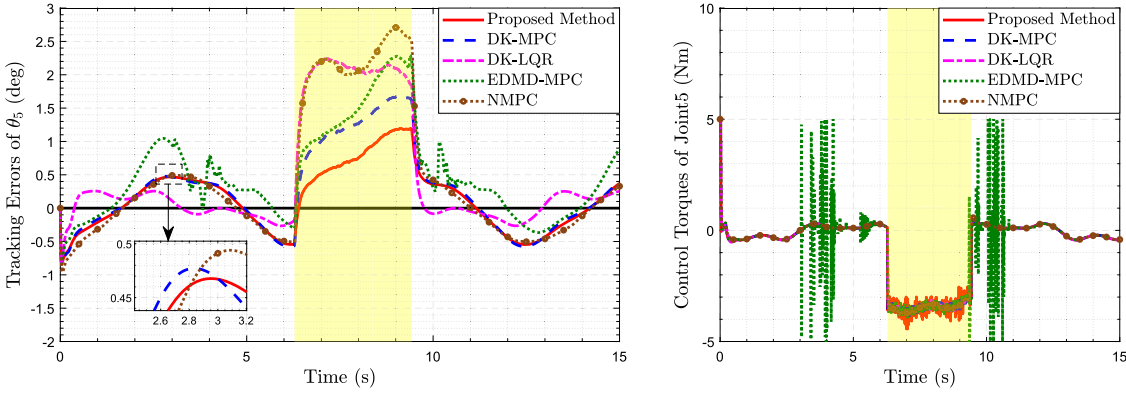


Fig. 13. Tracking errors and control torques of Joint5.

(IV). MSE, EV, and SMAE are used to evaluate control error, whereas ISCI and IV are used to measure control consumption.

The errors are defined as:  $e_i(k) = X_{ref}^{\theta_i}(k) - \theta_i(k)$  and  $e(k) = X_{ref}^{\theta}(k) - \theta(k)$ .

The indices MSE, EV, SMAE, ISCI, and IV are defined as follows:

$$MSE = \frac{\sum_{k=1}^{T/\Delta T} \|e(k)\|_2^2}{N}$$

$$EV = \frac{\sum_{k=1}^{T/\Delta T} \|e(k) - \mu_e\|_2^2}{N}$$

$$SMAE = \frac{1}{N} \sum_{i=1}^5 \max |e_i(k)| \quad (41)$$

$$ISCI = \sum_{k=1}^{T/\Delta T} \|u(k)\|_2^2$$

$$IV = \frac{\sum_{k=1}^{T/\Delta T} \|u(k) - \mu_u\|_2^2}{N}$$

where  $\mu_e$  denotes the average of  $e(k)$ ,  $\mu_u$  denotes the average of  $u(k)$ ,  $N = T/\Delta T$ .

Additionally, we compared the minimum and maximum improvements of the proposed method with those of four benchmark methods across these metrics.

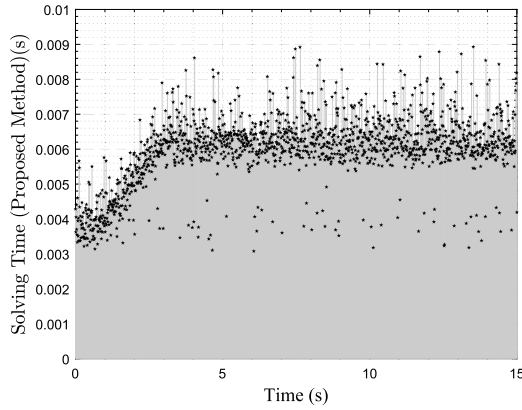
The control performances of the four controllers are shown in Table 6 and Table 7. Table 6 presents the control results for the interval  $(0, 2\pi]$ s, evaluating the tracking performance of the control system. Ta-

**Table 6**  
Tracking performance comparison.

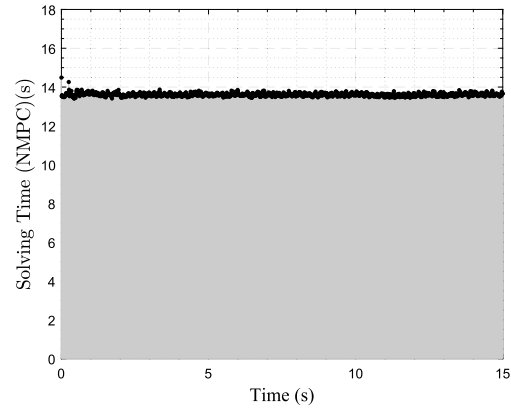
| $T \in (0, 2\pi]$ s  | MSE<br>$\frac{\sum_{k=1}^{T/\Delta T} \ e(k)\ _2^2}{N}$ | EV<br>$\frac{\sum_{k=1}^{T/\Delta T} \ e(k) - \mu_e\ _2^2}{N}$ | SMAE<br>$\sum_{i=1}^5 \max  e_i(k) $ | ISCI<br>$\sum_{k=1}^{T/\Delta T} \ u(k)\ _2^2$ | IV<br>$\frac{\sum_{k=1}^{T/\Delta T} \ u(k) - \mu_u\ _2^2}{N}$ |
|----------------------|---------------------------------------------------------|----------------------------------------------------------------|--------------------------------------|------------------------------------------------|----------------------------------------------------------------|
| Proposed Method      | $1.0274 \times 10^{-3}$                                 | $9.7986 \times 10^{-4}$                                        | 0.3138                               | $3.4883 \times 10^3$                           | $4.0132 \times 10^4$                                           |
| DK-MPC               | $1.0383 \times 10^{-3}$                                 | $9.8652 \times 10^{-4}$                                        | 0.3138                               | $3.4635 \times 10^3$                           | <b><math>4.0123 \times 10^4</math></b>                         |
| DK-LQR               | $1.0636 \times 10^{-3}$                                 | $9.9325 \times 10^{-4}$                                        | <b>0.3129</b>                        | $3.4970 \times 10^3$                           | $4.0766 \times 10^4$                                           |
| EDMD-MPC             | $1.1840 \times 10^{-3}$                                 | $1.1399 \times 10^{-3}$                                        | 0.3168                               | $8.1480 \times 10^3$                           | $4.0717 \times 10^4$                                           |
| NMPC                 | $1.1857 \times 10^{-3}$                                 | $1.1251 \times 10^{-3}$                                        | 0.3135                               | <b><math>3.0346 \times 10^3</math></b>         | $4.0364 \times 10^4$                                           |
| Improvement<br>(min) | 1.06%<br>(DK-MPC)                                       | 6.80%<br>(DK-MPC)                                              | -2.86%<br>(DK-LQR)                   | -13.01%<br>(NMPC)                              | -0.024%<br>(DK-MPC)                                            |
| Improvement<br>(max) | 15.41%<br>(NMPC)                                        | 16.33%<br>(EDMD-MPC)                                           | 9.56%<br>(EDMD-MPC)                  | 133.58%<br>(EDMD-MPC)                          | 1.58%<br>(DK-LQR)                                              |

**Table 7**  
Robustness performance comparison.

| $T \in (2\pi, 15]$ s | MSE<br>$\frac{\sum_{k=1}^{T/\Delta T} \ e(k)\ _2^2}{N}$ | EV<br>$\frac{\sum_{k=1}^{T/\Delta T} \ e(k) - \mu_e\ _2^2}{N}$ | SMAE<br>$\sum_{i=1}^5 \max  e_i(k) $ | ISCI<br>$\sum_{k=1}^{T/\Delta T} \ u(k)\ _2^2$ | IV<br>$\frac{\sum_{k=1}^{T/\Delta T} \ u(k) - \mu_u\ _2^2}{N}$ |
|----------------------|---------------------------------------------------------|----------------------------------------------------------------|--------------------------------------|------------------------------------------------|----------------------------------------------------------------|
| Proposed Method      | <b><math>4.4278 \times 10^{-4}</math></b>               | <b><math>3.8791 \times 10^{-4}</math></b>                      | <b>0.0986</b>                        | $2.1718 \times 10^4$                           | $8.4720 \times 10^6$                                           |
| DK-MPC               | $7.2645 \times 10^{-4}$                                 | $5.8694 \times 10^{-4}$                                        | 0.1228                               | <b><math>2.1634 \times 10^4</math></b>         | <b><math>8.4320 \times 10^6</math></b>                         |
| DK-LQR               | $1.9126 \times 10^{-3}$                                 | $1.2577 \times 10^{-3}$                                        | 0.1852                               | $2.1824 \times 10^4$                           | $8.4805 \times 10^6$                                           |
| EDMD-MPC             | $9.4795 \times 10^{-4}$                                 | $6.3635 \times 10^{-4}$                                        | 0.1373                               | $2.6488 \times 10^4$                           | $8.5244 \times 10^6$                                           |
| NMPC                 | $4.1843 \times 10^{-3}$                                 | $2.8962 \times 10^{-3}$                                        | 0.2814                               | $2.2098 \times 10^4$                           | $8.5414 \times 10^6$                                           |
| Improvement<br>(min) | 64.07%<br>(DK-MPC)                                      | 51.31%<br>(DK-MPC)                                             | 25.54%<br>(DK-MPC)                   | -0.39%<br>(DK-MPC)                             | -0.47%<br>(DK-MPC)                                             |
| Improvement<br>(max) | 845.01%<br>(NMPC)                                       | 646.62%<br>(NMPC)                                              | 185.4%<br>(NMPC)                     | 21.96%<br>(EDMD-MPC)                           | 0.82%<br>(NMPC)                                                |



(a) Optimization Time of Proposed Method



(b) Optimization Time of NMPC

**Fig. 14.** Optimization problem-solving time of two methods.

ble 7 presents the control results for the interval  $(2\pi, 15]$ s, assessing the robustness of the control system.

Furthermore, we analyze the optimization problem-solving time required by the proposed method and NMPC under the Matlab environment on the same computer, shown in Fig. 14.

#### 5.4. Suppression of base satellite attitude disturbance evaluation

In this subsection, we further evaluate the advantages of the proposed method in suppressing base satellite attitude disturbance. We compare the proposed method with: regardless of the treatment of base satellite attitude disturbance (proposed method without  $Q_2$ ), optimizing base satellite attitude disturbance without incorporating the decay factor  $\Gamma(k)$  (proposed method without  $\Gamma(k)$ ) and DK-LQR. Consider a more general scenario, the initial joint angles may not align with the desired trajectory. Initial joint angles and angular velocities are set to be

$\theta_i(0) = 90^\circ (i = 1, 2, \dots, 5)$ ,  $\dot{\theta}_i(0) = 0^\circ/\text{s} (i = 1, 2, \dots, 5)$ . The desired tracking trajectory  $X_{ref}$  is the same as specified in Subsection 5.3.

The simulation results of the tracking errors and control torques of each joint are shown in Figs. 15 to 19. Besides, we use  $\|\omega_0\|_2^2$  to represent the base satellite attitude disturbance. The base satellite attitude disturbances and changes of  $\Gamma(k)$  are shown in Fig. 20.

In addition to demonstrating the control performances, we use the indices: MSE, EV, ISCI, IV and integral base satellite attitude disturbance (IBSAD).

The IBSAD is defined as follows:

$$\text{IBSAD} = \sum_{k=1}^{T/\Delta T} \|\omega_0(k)\|_2^2 \quad (42)$$

## 6. Simulation results analyses and discussions

This section presents analyses and discussions of the simulation results presented in Section 5.



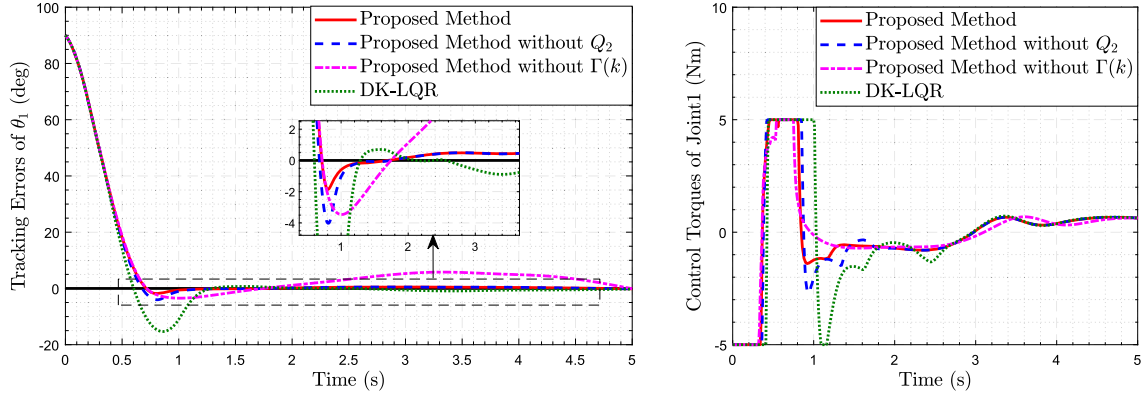


Fig. 15. Tracking errors and control torques of Joint1.

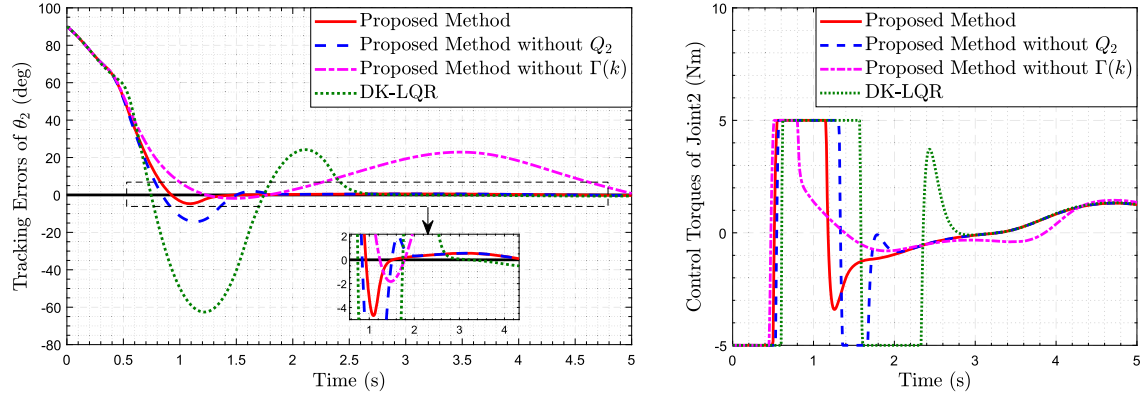


Fig. 16. Tracking errors and control torques of Joint2.

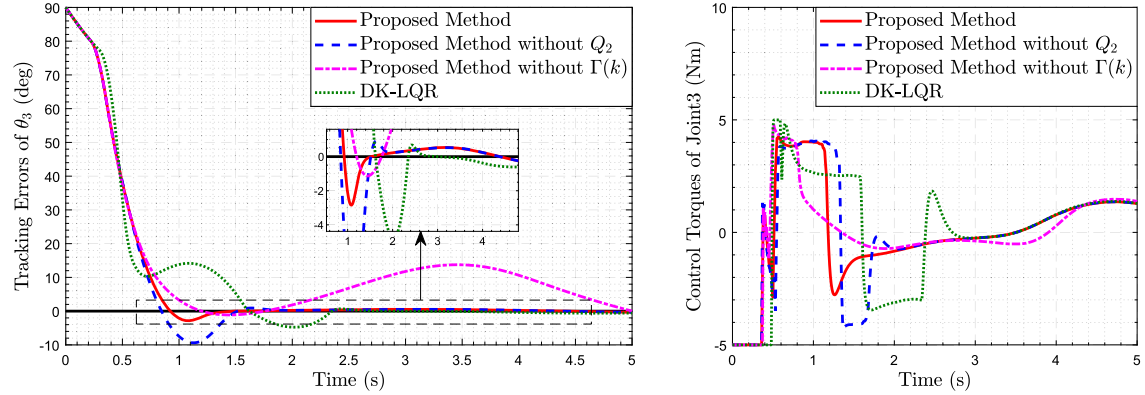


Fig. 17. Tracking errors and control torques of Joint3.

### 6.1. Training and prediction evaluation results

Fig. 6 shows the rapid convergence of the DNN training loss, indicating that the observable functions are learned by the DNN.

Fig. 7 highlights the significance of Assumption 4. In some studies that utilize the DK solely for prediction [49,50], the presence of biases enhances the expressive capability of DNN. However, this paper focuses on control issues. If biases are not set to zero, the controlled system may experience undesirable state drift at the equilibrium points. This is because even if the states converge to zero, the values of the observable functions might not be zero, leading to continuous computation of control inputs and potentially causing system divergence. For simple systems, extensive data and thorough training can mitigate the effects of not setting biases to zero. Nonetheless, this paper directly avoids this potential problem by explicitly setting biases to zero.

It is evident from Fig. 8 and Table 5 that the predictive performance of DK significantly surpasses that of EDMD. This can be attributed to the fact that EDMD relies on manually selected observable functions, which inherently possess limitations, whereas DK utilizes DNN trained to derive observable functions. Consequently, DK is capable of capturing rich mapping relationships of observable functions, thereby exhibiting superior predictive performance compared to EDMD.

### 6.2. Tracking precision and robustness results

Figs. 9 to 13, along with Tables 6 and 7, clearly illustrate that the proposed method excels in both precision and robustness.

In terms of tracking performance, the proposed method shows advantages over the four other methods in both MSE and EV. Specifically, the proposed method achieves a 1.06% improvement in MSE and

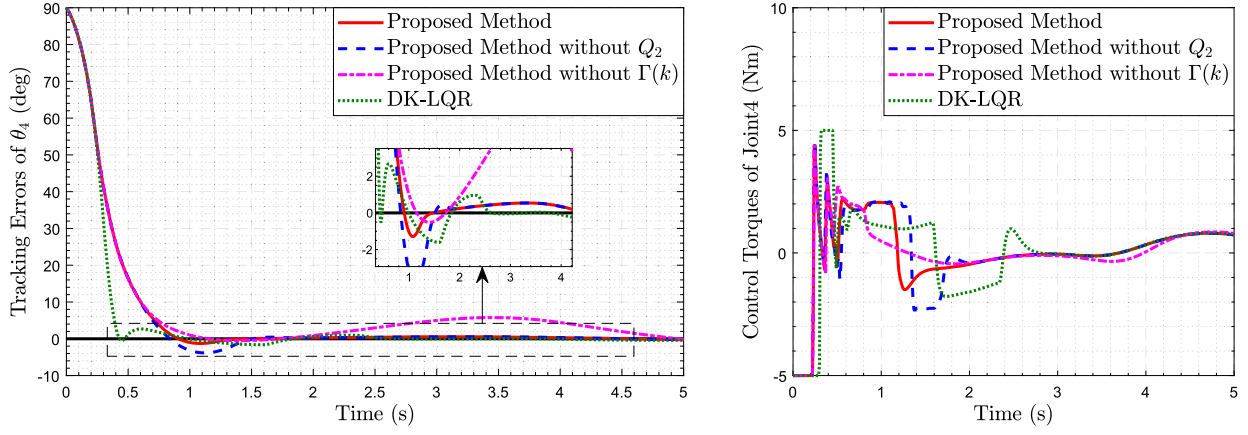


Fig. 18. Tracking errors and control torques of Joint4.

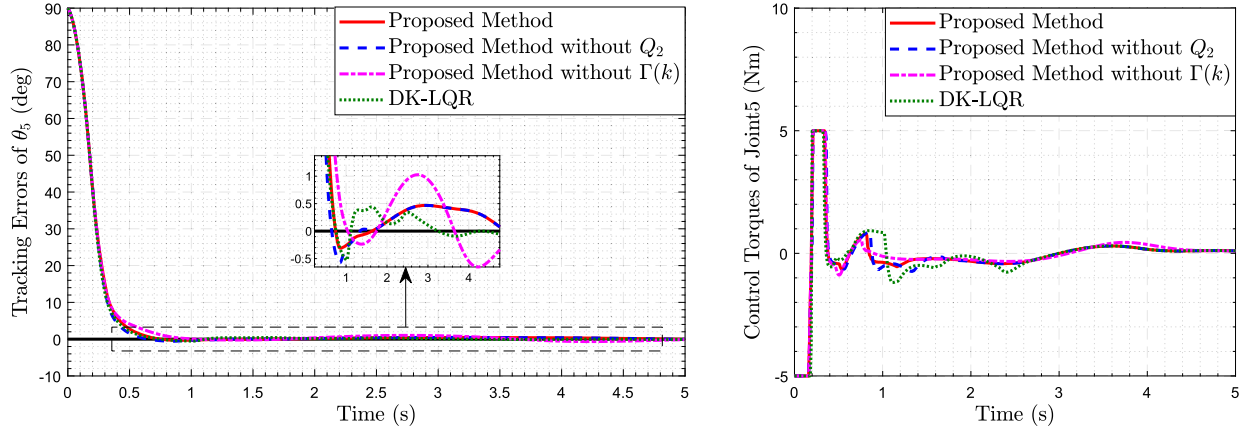


Fig. 19. Tracking errors and control torques of Joint5.

a 6.8% enhancement in EV compared to DK-MPC. This improvement is attributed to the use of RBFNN-DO for observing modeling errors, which enhances tracking accuracy. DK-MPC exhibits a slight improvement in both MSE and EV over DK-LQR, primarily due to the larger control inputs at the initial stage of tracking, which DK-LQR is unable to handle input constraints. This phenomenon will be further explained in the next subsection. However, DK-LQR performs best in the SMAE. EDMD-MPC suffers from significant control input chattering due to inappropriate dynamic linearization, resulting in poor tracking performance. The proposed method exhibits a 16.33% improvement in EV compared to EDMD-MPC, while its ISCI demonstrates a 133.58% enhancement in comparison to EDMD-MPC. While NMPC requires an accurate nonlinear dynamic model, the proposed method improves MSE by 15.41% compared to NMPC. However, NMPC excels the best performance in terms of energy consumption, with the proposed method's ISCI lagging by 13.01%.

In terms of robustness performance, the proposed method demonstrates significant advantages compared to the four other methods. When faced with disturbances, the proposed method achieves a 64.07% improvement in MSE, a 51.31% enhancement in EV, and a 25.52% improvement in SMAE over DK-MPC. DK-MPC shows improvement in MSE, EV, and SMAE over DK-LQR because MPC's utilization of rolling optimization enhances controller robustness, surpassing LQR's performance. Compared to NMPC, the proposed method's MSE improves by 845.01%, EV improves by 646.62%, and SMAE improves by 185.4%. This is because NMPC relies on an accurate model, which makes it less robust compared to the proposed method. In terms of energy consumption, the proposed method has a slight disadvantage compared to DK-MPC. However, it outperforms the other methods because the proposed method

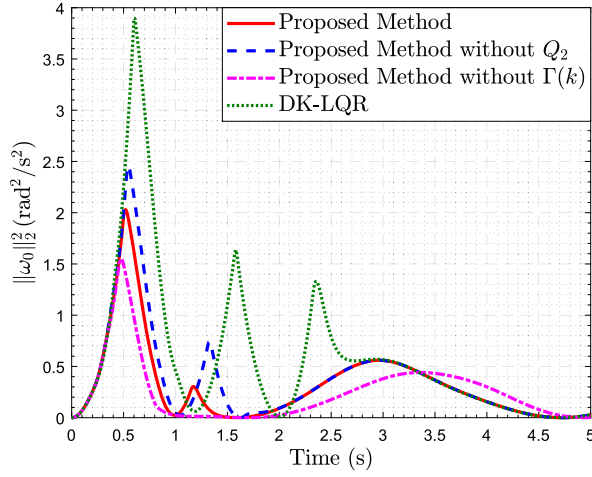
experiences smaller state variations when subjected to disturbances, requiring less energy to return to equilibrium.

From Fig. 14, it is evident that due to the need to solve a complex nonlinear optimization problem within the prediction horizon, NMPC requires a substantial computational effort. In contrast, the proposed method entails solving a simpler quadratic programming problem within the prediction horizon. Consequently, the computational burden of the proposed method is significantly lower compared to NMPC in terms of optimization problem-solving time, indicating its potential for practical applications.

### 6.3. Suppression of base satellite attitude disturbance results

Based on Figs. 15 to 20 and Table 8, it is evident that the proposed method demonstrates superior performance in the suppression of base satellite attitude disturbance without loss tracking precision.

The mutual coupling between the base satellite and the manipulator of the free-floating space robot implies that the actuations of manipulator joint angles generate disturbances on the base satellite, which, in turn, affects the dynamic characteristics during joint angle tracking. Owing to the proposed method's consideration for the suppression of base satellite attitude disturbance during the control process, the joint angles exhibit a smoother approach toward the desired trajectory compared to the proposed method without  $Q_2$ , thereby reducing overshoot at the beginning of the control process. Although the proposed method without  $\Gamma(k)$  excels in EV, IBSAD, ISCI and IV, the trade-off is a significantly lower accuracy in the tracking phase compared to the proposed method. Due to the introduction of the decay factor, the proposed method harmonizes the strengths and weaknesses of the proposed method without



(a) Base satellite Attitude Disturbance

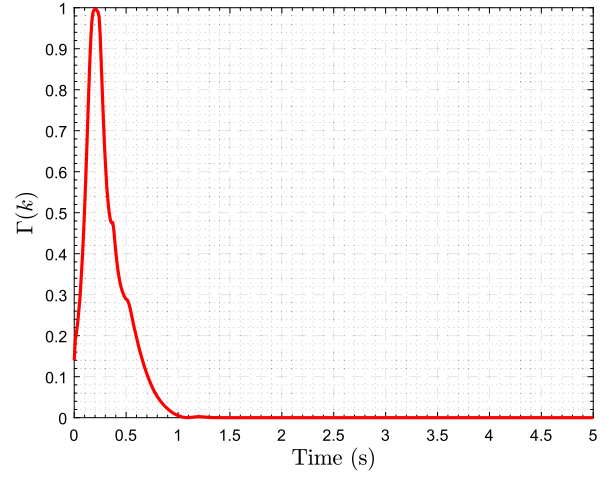
(b) Change of  $\Gamma(k)$ Fig. 20. Base satellite attitude disturbance of four controllers and change of  $\Gamma(k)$ .

Table 8

Performance comparison.

|                                        | MSE<br>$\frac{\sum_{k=1}^{T/\Delta T} \ e(k)\ _2^2}{N}$ | EV<br>$\frac{\sum_{k=1}^{T/\Delta T} \ e(k) - \mu_e\ _2^2}{N}$ | IBSAD<br>$\sum_{k=1}^{T/\Delta T} \ \omega_0(k)\ _2^2$ | ISCI<br>$\sum_{k=1}^{T/\Delta T} \ u(k)\ _2^2$ | IV<br>$\frac{\sum_{k=1}^{T/\Delta T} \ u(k) - \mu_u\ _2^2}{N}$ |
|----------------------------------------|---------------------------------------------------------|----------------------------------------------------------------|--------------------------------------------------------|------------------------------------------------|----------------------------------------------------------------|
| Proposed Method                        | <b>0.6658</b>                                           | 0.5897                                                         | 123.8397                                               | $9.5713 \times 10^3$                           | $3.6757 \times 10^3$                                           |
| Proposed Method<br>without $Q_2$       | 0.6675                                                  | 0.6025                                                         | 173.8321                                               | $1.1598 \times 10^4$                           | $4.3144 \times 10^3$                                           |
| Proposed Method<br>without $\Gamma(k)$ | 0.7395                                                  | <b>0.5345</b>                                                  | <b>73.9641</b>                                         | <b><math>7.6499 \times 10^3</math></b>         | <b><math>6.1120 \times 10^2</math></b>                         |
| DK-LQR                                 | 0.8082                                                  | 0.7599                                                         | 457.2700                                               | $1.5068 \times 10^4$                           | $6.9374 \times 10^3$                                           |

$Q_2$  and the proposed method without  $\Gamma(k)$ , suppressing base satellite attitude disturbance during the initial control phase to enhance dynamic characteristics, while prioritizing tracking precision during the trajectory tracking control phase. Compared to DK-LQR, the proposed method significantly outperforms in dynamic characteristics as LQR cannot handle input constraints.

## 7. Conclusions

This paper initially presents an approach that leverages Koopman operator and DNN for modeling the nonlinear dynamics of micro-nano free-floating space robots. This approach offers a significant reduction in computational burden by facilitating MPC based on a globally linearized model compared to NMPC. A key innovation lies in integrating a decay factor within the MPC optimization objective function to balance joint angle tracking precision and the suppression of base satellite attitude disturbance. Further enhancing the control strategy, an RBFNN-DO is implemented for online compensation of modeling inaccuracies and external disturbances, thereby improving tracking precision and robustness. The simulation results underscore the superior tracking precision, robustness against disturbances, and computational efficiency of the proposed method compared to traditional NMPC and other baseline methods. Particularly, the proposed method exhibits enhanced capabilities in suppressing base satellite attitude disturbance without compromising tracking precision. This research contributes to advancing the control strategy of free-floating space robots, highlighting the potential of learning-based methodologies in overcoming the challenges posed by their complex dynamics and interaction with the environment.

However, this paper has some limitations. Firstly, although the proposed method offers numerous advantages over traditional NMPC, it still lags behind in energy consumption during the tracking phase, and the tracking accuracy can be further improved. Therefore, future work could focus on integrating the proposed method with other intelligent

approaches to further reduce control energy consumption and enhance tracking accuracy. Additionally, although one advantage of the proposed method in this paper is its rapid optimization control solving speed, no experiments have been conducted on flight hardware. Future experiments could be conducted to validate the real-time performance of the proposed method.

## CRediT authorship contribution statement

**Renhao Mao:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Tao Meng:** Supervision, Project administration, Funding acquisition. **Kun Wang:** Visualization, Validation, Formal analysis, Conceptualization. **Jiakun Lei:** Writing – original draft, Validation, Conceptualization. **Weijia Wang:** Validation, Formal analysis, Conceptualization.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

Data will be made available on request.

## Acknowledgement

This research was supported by Zhejiang Provincial Natural Science Foundation of China under Grant No. LR24F030001.

The authors would like to thank the handling Associate Editor and the anonymous reviewers for their insightful comments, which help to improve the quality of our article.

## Appendix A. Proof of Theorem 1

Define the estimation error of integrated disturbances as  $\tilde{\mathbf{D}}(k) = \hat{\mathbf{D}}(k) - \mathbf{D}(k)$ . Then, choose a candidate Lyapunov function as follows:

$$V(k) = V_1(k) + V_2(k) + V_3(k) \quad (\text{A.1})$$

where:

$$V_1(k) = \|\xi(k)\|_2^2, \quad V_2(k) = \frac{1}{\eta} \sum_{i=1}^{n_z} \|\tilde{\mathbf{W}}_i(k)\|_2^2, \quad V_3(k) = \|\tilde{\mathbf{D}}(k)\|_2^2 \quad (\text{A.2})$$

and  $\tilde{\mathbf{W}}_i(k) \in \mathbb{R}^l$  represents the  $i$ -th column estimation error of  $\mathbf{W}(k)$ .

The first difference of  $V_1(k)$  is given by:

$$\begin{aligned} \Delta V_1(k) &= V_1(k+1) - V_1(k) \\ &= \|\xi(k+1)\|_2^2 - \|\xi(k)\|_2^2 \end{aligned} \quad (\text{A.3})$$

The first difference of  $V_2(k)$  is given by:

$$\begin{aligned} \Delta V_2(k) &= V_2(k+1) - V_2(k) \\ &= \frac{1}{\eta} \sum_{i=1}^{n_z} \|\tilde{\mathbf{W}}_i(k+1)\|_2^2 - \frac{1}{\eta} \sum_{i=1}^{n_z} \|\tilde{\mathbf{W}}_i(k)\|_2^2 \\ &= \sum_{i=1}^{n_z} \left[ \eta \xi_i^2(k+1) \|\mathbf{G}(\mathbf{z}(k))\|_2^2 - 2\gamma \xi_i(k+1) \mathbf{G}^\top(\mathbf{z}(k)) \tilde{\mathbf{W}}_i(k) \right. \\ &\quad \left. - 2\xi_i(k+1) \tilde{\mathbf{D}}_i^\top(k) - 2\xi_i^2(k+1) + \frac{\gamma^2}{\eta} \|\tilde{\mathbf{W}}_i(k)\|_2^2 \right. \\ &\quad \left. - \frac{2\gamma}{\eta} \tilde{\mathbf{W}}_i^\top(k) \tilde{\mathbf{W}}_i(k) \right] \end{aligned} \quad (\text{A.4})$$

where  $\xi_i(k+1)$  is the  $i$ -th element of  $\xi(k+1)$ ,  $\tilde{\mathbf{D}}_i(k)$  is the  $i$ -th element of  $\tilde{\mathbf{D}}(k)$ .

Thus:

$$\begin{aligned} \Delta V_1(k) + \Delta V_2(k) &= -\|\xi(k+1)\|_2^2 - \|\xi(k)\|_2^2 - 2\xi^\top(k+1) \tilde{\mathbf{D}}(k) \\ &\quad + \sum_{i=1}^{n_z} \left[ \eta \xi_i^2(k+1) \|\mathbf{G}(\mathbf{z}(k))\|_2^2 \right. \\ &\quad \left. - 2\gamma \xi_i(k+1) \mathbf{G}^\top(\mathbf{z}(k)) \tilde{\mathbf{W}}_i(k) \right. \\ &\quad \left. - \frac{\gamma^2}{\eta} \|\tilde{\mathbf{W}}_i(k)\|_2^2 - \frac{2\gamma}{\eta} \tilde{\mathbf{W}}_i^\top(k) \tilde{\mathbf{W}}_i(k) \right] \end{aligned} \quad (\text{A.5})$$

Considering:

$$\begin{aligned} \|\mathbf{G}(\mathbf{z}(k))\|_2^2 &< l, \\ \frac{2\gamma}{\eta} \tilde{\mathbf{W}}_i^\top(k) \tilde{\mathbf{W}}_i(k) &= \frac{\gamma}{\eta} \|\tilde{\mathbf{W}}_i(k)\|_2^2 + \frac{\gamma}{\eta} \|\tilde{\mathbf{W}}_i(k)\|_2^2 - \frac{\gamma}{\eta} \|\mathbf{W}^*(k)\|_2^2 \end{aligned} \quad (\text{A.6})$$

Thus:

$$\begin{aligned} \Delta V_1(k) + \Delta V_2(k) &\leq -\|\xi(k+1)\|_2^2 - \|\xi(k)\|_2^2 - 2\xi^\top(k+1) \tilde{\mathbf{D}}(k) \\ &\quad + \eta l \|\xi(k+1)\|_2^2 + \frac{\gamma^2}{\eta} \|\mathbf{W}^*(k)\|_2^2 - \frac{\gamma}{\eta} \|\tilde{\mathbf{W}}(k)\|_2^2 \\ &\quad - \frac{\gamma}{\eta} \|\tilde{\mathbf{W}}(k)\|_2^2 + \frac{\gamma}{\eta} \|\mathbf{W}^*(k)\|_2^2 \\ &\quad - 2\gamma \sum_{i=1}^{n_z} \xi_i(k+1) \mathbf{G}^\top(\mathbf{z}(k)) \tilde{\mathbf{W}}_i(k) \end{aligned} \quad (\text{A.7})$$

The first difference of  $V_3(k)$  is given by:

$$\begin{aligned} \Delta V_3(k) &= V_3(k+1) - V_3(k) \\ &= \|\tilde{\mathbf{D}}(k+1)\|_2^2 - \|\tilde{\mathbf{D}}(k)\|_2^2 \\ &= \|\mathbf{D}(k+1)\|_2^2 - 2\alpha \mathbf{D}^\top(k+1) \tilde{\mathbf{W}}(k) \mathbf{G}(\mathbf{z}(k)) \\ &\quad + 2\alpha \mathbf{D}^\top(k+1) \tilde{\mathbf{D}}(k) + \alpha^2 \|\mathbf{G}^\top(\mathbf{z}(k)) \tilde{\mathbf{W}}(k)\|_2^2 \\ &\quad - 2\alpha^2 \mathbf{G}^\top(\mathbf{z}(k)) \tilde{\mathbf{W}}(k) \tilde{\mathbf{D}}(k) + (\alpha^2 - 1) \|\tilde{\mathbf{D}}(k)\|_2^2 \end{aligned} \quad (\text{A.8})$$

According to Young's inequality [60]:

$$\begin{aligned} -2\xi^\top(k+1) \tilde{\mathbf{D}}(k) &\leq \rho_1^2 \|\xi(k+1)\|_2^2 + \frac{1}{\rho_1^2} \|\tilde{\mathbf{D}}(k)\|_2^2 \\ -2\gamma \sum_{i=1}^{n_z} \xi_i(k+1) \mathbf{G}^\top(\mathbf{z}(k)) \tilde{\mathbf{W}}_i(k) &\leq \gamma^2 \rho_2^2 l \|\xi(k+1)\|_2^2 + \frac{1}{\rho_2^2} \|\tilde{\mathbf{W}}(k)\|_2^2 \\ -2\alpha \mathbf{D}^\top(k+1) \tilde{\mathbf{W}}(k) \mathbf{G}(\mathbf{z}(k)) &\leq \rho_3^2 \|\mathbf{D}(k+1)\|_2^2 + \frac{\alpha^2 l}{\rho_3^2} \|\tilde{\mathbf{W}}(k)\|_2^2 \\ 2\alpha \mathbf{D}^\top(k+1) \tilde{\mathbf{D}}(k) &\leq \rho_4^2 \|\mathbf{D}(k+1)\|_2^2 + \frac{\alpha^2}{\rho_4^2} \|\tilde{\mathbf{D}}(k)\|_2^2 \\ -2\alpha^2 \mathbf{G}^\top(\mathbf{z}(k)) \tilde{\mathbf{W}}(k) \tilde{\mathbf{D}}(k) &\leq \rho_5^2 \|\tilde{\mathbf{D}}(k)\|_2^2 + \frac{\alpha^4 l^2}{\rho_5^2} \|\tilde{\mathbf{W}}(k)\|_2^2 \end{aligned} \quad (\text{A.9})$$

where  $\rho_1, \rho_2, \rho_3, \rho_4, \rho_5$  are non-zero constants.

Thus:

$$\begin{aligned} \Delta V(k) &= \Delta V_1(k) + \Delta V_2(k) + \Delta V_3(k) \\ &\leq -\left(1 - \rho_1^2 - l\eta - l\gamma^2 \rho_2^2\right) \|\xi(k+1)\|_2^2 - \|\xi(k)\|_2^2 + \frac{\gamma}{\eta} \|\mathbf{W}^*(k)\|_2^2 \\ &\quad - \left(\frac{\gamma}{\eta} - \alpha^2 l^2 - \frac{\alpha^2 l}{\rho_3^2} - \frac{\alpha^4 l^2}{\rho_5^2}\right) \|\tilde{\mathbf{W}}(k)\|_2^2 \\ &\quad - \left(\frac{\gamma}{\eta} - \frac{\gamma^2}{\eta} - \frac{1}{\rho_2^2}\right) \|\tilde{\mathbf{W}}(k)\|_2^2 \\ &\quad - \left(1 - \frac{1}{\rho_1^2} - \alpha^2 - \rho_3^2 - \frac{\alpha^2}{\rho_4^2}\right) \|\tilde{\mathbf{D}}(k)\|_2^2 \\ &\quad + (1 + \rho_3^2 + \rho_4^2) \|\mathbf{D}(k+1)\|_2^2 \end{aligned} \quad (\text{A.10})$$

Let  $C_0 = \frac{\gamma}{\eta} \|\mathbf{W}^*(k)\|_2^2 + (1 + \rho_3^2 + \rho_4^2) \|\mathbf{D}(k+1)\|_2^2$ ,  $\Delta V(k) < 0$  holds as long as the following condition holds:

$$\begin{aligned} \|\xi(k+1)\|_2 &> \sqrt{\frac{C_0}{1 - \rho_1^2 - l\eta - l\gamma^2 \rho_2^2}}, \\ \|\tilde{\mathbf{D}}(k)\|_2 &> \sqrt{\frac{C_0}{1 - \frac{1}{\rho_1^2} - \alpha^2 - \rho_3^2 - \frac{\alpha^2}{\rho_4^2}}}, \\ \|\tilde{\mathbf{W}}(k)\|_2 &> \sqrt{\frac{C_0}{\frac{\gamma}{\eta} - \alpha^2 l^2 - \frac{\alpha^2 l}{\rho_3^2} - \frac{\alpha^4 l^2}{\rho_5^2}}}, \quad \|\tilde{\mathbf{W}}(k)\|_2 > \sqrt{\frac{C_0}{\frac{\gamma}{\eta} - \frac{\gamma^2}{\eta} - \frac{1}{\rho_2^2}}} \end{aligned} \quad (\text{A.11})$$

Invoking the Lemma 1, by selecting appropriate  $l, \eta, \alpha, \gamma$ , the estimation error of internal and external disturbances are UUB.

## Appendix B. Proof of Theorem 3

Define a Lyapunov function:

$$V_4(k) = J^*(k) \quad (\text{B.1})$$

where  $J^*(k)$  is the optimal objective cost function.

Define the objective cost function of the feasible trajectory at time  $k+1$ :  $\bar{J}(k+1)$ .



The difference of  $\bar{J}(k+1)$  and  $J^*(k)$  is:

$$\begin{aligned}\Delta J(k) &= \bar{J}(k+1) - J^*(k) \\ &= \sum_{i=0}^{N_p-1} [L(\hat{z}(k+i+1|k+1), u(k+i+1|k+1)) \\ &\quad - L(\hat{z}(k+i|k), u^*(k+i|k))] \\ &\quad + L_f(\hat{z}(k+N_p+1|k+1)) - L_f(\hat{z}(k+N_p|k)) \\ &= \sum_{i=0}^{N_p-2} [L(\hat{z}(k+i+1|k+1), u(k+i+1|k+1)) \\ &\quad - L(\hat{z}(k+i+1|k), u^*(k+i+1|k))] \\ &\quad + L(\hat{z}(k+N_p|k+1), u(k+N_p|k+1)) \\ &\quad - L(\hat{z}(k), u^*(k)) + L_f(\hat{z}(k+N_p+1|k+1)) \\ &\quad - L_f(\hat{z}(k+N_p|k)) + L_f(\hat{z}(k+N_p|k+1)) \\ &\quad - L_f(\hat{z}(k+N_p|k+1))\end{aligned}\quad (B.2)$$

Invoking Assumption 6 and Assumption 7:

$$\begin{aligned}&L(\hat{z}(k+i+1|k+1), u(k+i+1|k+1)) - L(\hat{z}(k+i+1|k), u^*(k+i+1|k)) \\ &\leq L_a A^i Y \\ &L_f(\hat{z}(k+N_p|k+1)) - L_f(\hat{z}(k+N_p|k)) \leq L_b A^{N_p-1} Y\end{aligned}\quad (B.3)$$

Invoking the Assumption 8:

$$\begin{aligned}\Delta J(k) &\leq L(\hat{z}(k+N_p|k+1), u(k+N_p|k+1)) + L_f(\hat{z}(k+N_p+1|k+1)) \\ &\quad - L_f(\hat{z}(k+N_p|k+1)) + \left( L_b A^{N_p-1} + L_a \frac{A^{N_p-1} - I_{n_z}}{A - I_{n_z}} \right) Y \\ &\quad - L(\hat{z}(k), u^*(k)) \\ &\leq L_c Y - L(\hat{z}(k), u^*(k))\end{aligned}\quad (B.4)$$

where  $L_c = L_b A^{N_p-1} + L_a \frac{A^{N_p-1} - I_{n_z}}{A - I_{n_z}}$ .

Invoking Assumption 6,  $L(\hat{z}(k), u^*(k)) \geq a_1 \|z(k)\|_2$ , the first difference of Equation (B.1) is:

$$\Delta V_4(k) = J^*(k+1) - J^*(k) \leq \Delta J(k) \leq L_c Y - a_1 \|z(k)\|_2 \quad (B.5)$$

Invoking Lemma 2, the proposed control strategy is ISS.

## References

- [1] W.-J. Li, D.-Y. Cheng, X.-G. Liu, Y.-B. Wang, W.-H. Shi, Z.-X. Tang, F. Gao, F.-M. Zeng, H.-Y. Chai, W.-B. Luo, Q. Cong, Z.-L. Gao, On-orbit service (OOS) of spacecraft: a review of engineering developments, *Prog. Aerosp. Sci.* 108 (2019) 32–120, <https://doi.org/10.1016/j.paerosci.2019.01.004>.
- [2] Y. Liu, Z. Jin, L. Teng, PSO-based time optimal rapid orientation for micronano space robot, *IEEE Trans. Aerosp. Electron. Syst.* 59 (2) (2022) 1921–1934, <https://doi.org/10.1109/TAES.2022.3207124>.
- [3] X. Shao, G. Sun, C. Xue, X. Li, Nonsingular terminal sliding mode control for free-floating space manipulator with disturbance, *Acta Astronaut.* 181 (2021) 396–404, <https://doi.org/10.1016/j.actaastro.2021.01.038>.
- [4] Q. Yao, Robust finite-time trajectory tracking control for a space manipulator with parametric uncertainties and external disturbances, *Proc. Inst. Mech. Eng., G J. Aerosp. Eng.* 236 (2) (2022) 396–409, <https://doi.org/10.1177/09544100211014754>.
- [5] J. Lavín-Delgado, S. Chávez-Vázquez, J. Gómez-Aguilar, M.O. Alassafi, F.E. Alsaadi, A.M. Ahmad, Intelligent neural integral sliding-mode controller for a space robotic manipulator mounted on a free-floating satellite, *Adv. Space Res.* 71 (9) (2023) 3734–3747, <https://doi.org/10.1016/j.asr.2022.08.053>.
- [6] B. Dou, X. Yue, Disturbance observer-based fractional-order sliding mode control for free-floating space manipulator with disturbance, *Aerosp. Sci. Technol.* 132 (2023) 108061, <https://doi.org/10.1109/ROBIO54168.2021.9739210>.
- [7] Q. Yao, Adaptive fuzzy neural network control for a space manipulator in the presence of output constraints and input nonlinearities, *Adv. Space Res.* 67 (6) (2021) 1830–1843, <https://doi.org/10.1016/j.asr.2021.01.001>.
- [8] A. Prakash, D.K. Giri, S.R. Kumar, Dynamic velocity error based trajectory tracking for space robotic manipulator, *Aerosp. Sci. Technol.* 126 (2022) 107650, <https://doi.org/10.1016/j.ast.2022.107650>.
- [9] Z. Xie, T. Sun, T.H. Kwan, Z. Mu, X. Wu, A new reinforcement learning based adaptive sliding mode control scheme for free-floating space robotic manipulator, *IEEE Access* 8 (2020) 127048–127064, <https://doi.org/10.1109/ACCESS.2020.3008399>.
- [10] R. Jin, P. Rocco, Y. Geng, Observer-based fixed-time tracking control for space robots in task space, *Acta Astronaut.* 184 (2021) 35–45, <https://doi.org/10.1016/j.actaastro.2021.04.002>.
- [11] R. Jin, Y. Geng, X. Chen, Predefined-time control for free-floating space robots in task space, *J. Franklin Inst.* 358 (18) (2021) 9542–9560, <https://doi.org/10.1016/j.jfranklin.2021.09.030>.
- [12] Y. Yan, H. Cui, P. Han, Fixed-time control for free-floating space manipulators with prescribed constraints and input saturation, *Aeronaut. J.* 128 (1324) (2024) 1219–1244, <https://doi.org/10.1017/aer.2023.99>.
- [13] T. Sands, Development of deterministic artificial intelligence for unmanned underwater vehicles (UUV), *Journal of Marine Science and Engineering* 8 (8) (2020) 578, <https://doi.org/10.3390/jmse8080578>.
- [14] E. Wilt, T. Sands, Microsatellite uncertainty control using deterministic artificial intelligence, *Sensors* 22 (22) (2022) 8723, <https://doi.org/10.3390/s22228723>.
- [15] T. Sands, Optimization provenance of whiplash compensation for flexible space robotics, *Aerospace* 6 (9) (2019) 93, <https://doi.org/10.3390/aerospace609093>.
- [16] T. Sands, Flattening the curve of flexible space robotics, *Appl. Sci.* 12 (6) (2022) 2992, <https://doi.org/10.3390/app12062992>.
- [17] F. Dörfler, P. Tesi, C. De Persis, On the certainty-equivalence approach to direct data-driven LQR design, *IEEE Trans. Autom. Control* 68 (12) (2023) 7989–7996, <https://doi.org/10.1109/TAC.2023.3253787>.
- [18] L. Yuan, L. Wang, J. Zhang, Adaptive dynamic programming base on MMC device of a flexible high-altitude long endurance aircraft, *Aerosp. Sci. Technol.* (2024) 109305, <https://doi.org/10.1016/j.ast.2024.109305>.
- [19] W.B. Greer, C. Sultan, Infinite horizon model predictive control tracking application to helicopters, *Aerosp. Sci. Technol.* 98 (2020) 105675, <https://doi.org/10.1016/j.ast.2019.105675>.
- [20] Z. Wang, P. Qi, N. Bai, X. Liu, Z. Jiao, A novel aircraft anti-skid brake control method based on multi-objective model predictive control for suppressing landing gear walk vibration, *Aerosp. Sci. Technol.* 132 (2023) 108062, <https://doi.org/10.1016/j.ast.2022.108062>.
- [21] Y. Wang, M. Pan, W. Zhou, J. Huang, Direct thrust control for variable cycle engine based on fractional order PID-nonlinear model predictive control under off-nominal operation conditions, *Aerosp. Sci. Technol.* 143 (2023) 108726, <https://doi.org/10.1016/j.ast.2023.108726>.
- [22] T. Rybus, K. Seweryn, J.Z. Sasiadek, Control system for free-floating space manipulator based on nonlinear model predictive control (NMPC), *J. Intell. Robot. Syst.* 85 (2017) 491–509, <https://doi.org/10.1007/s10846-016-0396-2>.
- [23] R. Srivastava, R. Sah, K. Das, Nonlinear model predictive control of rotation floating space robots for autonomous active debris removal, *IFAC-PapersOnLine* 55 (22) (2022) 147–152, <https://doi.org/10.1016/j.ifacol.2023.03.025>.
- [24] B.O. Koopman, Hamiltonian systems and transformation in Hilbert space, *Proc. Natl. Acad. Sci.* 17 (5) (1931) 315–318, <https://doi.org/10.1073/pnas.17.5.315>.
- [25] T. Chen, J. Shan, H. Wen, Koopman-operator-based attitude dynamics and control on SO (3), in: *Distributed Attitude Consensus of Multiple Flexible Spacecraft*, Springer, 2022, pp. 177–210.
- [26] J. Yao, Q. Hu, J. Zheng, Koopman-operator-based safe learning control for spacecraft attitude reorientation with angular velocity constraints, *IEEE Trans. Aerosp. Electron. Syst.* (2023), <https://doi.org/10.1109/TAES.2023.3285725>.
- [27] S. Servadio, R. Armellin, R. Linares, Koopman-operator control optimization for relative motion in space, *J. Guid. Control Dyn.* 46 (11) (2023) 2121–2132, <https://doi.org/10.2514/1.G007217>.
- [28] M. Korda, I. Mezić, Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control, *Automatica* 93 (2018) 149–160, <https://doi.org/10.1016/j.automatica.2018.03.046>.
- [29] H. Shi, M.Q.-H. Meng, Deep Koopman operator with control for nonlinear systems, *IEEE Robot. Autom. Lett.* 7 (3) (2022) 7700–7707, <https://doi.org/10.1109/LRA.2022.3184036>.
- [30] Y. Han, W. Hao, U. Vaidya, Deep learning of Koopman representation for control, in: *2020 59th IEEE Conference on Decision and Control (CDC)*, IEEE, 2020, pp. 1890–1895.
- [31] W. Hao, B. Huang, W. Pan, D. Wu, S. Mou, Deep Koopman learning of nonlinear time-varying systems, *Automatica* 159 (2024) 111372, <https://doi.org/10.1016/j.automatica.2023.111372>.
- [32] H. Chen, X. He, S. Cheng, C. Lv, Deep Koopman operator-informed safety command governor for autonomous vehicles, *IEEE/ASME Trans. Mechatron.* (2024) 1–11, <https://doi.org/10.1109/TMECH.2023.3349052>.
- [33] Y. Xiao, X. Zhang, X. Xu, X. Liu, J. Liu, Deep neural networks with Koopman operators for modeling and control of autonomous vehicles, *IEEE Transactions on Intelligent Vehicles* 8 (1) (2023) 135–146, <https://doi.org/10.1109/TIV.2022.3180337>.



- [34] J. Zhang, H. Wang, Online model predictive control of robot manipulator with structured deep Koopman model, *IEEE Robot. Autom. Lett.* 8 (5) (2023) 3102–3109, <https://doi.org/10.1109/TIV.2022.3180337>.
- [35] T. Zhao, M. Yue, J. Wang, Deep learning-based Koopman modeling for online control synthesis of nonlinear power system transient dynamics, *IEEE Trans. Ind. Inform.* 19 (10) (2023) 10444–10453, <https://doi.org/10.1109/TII.2023.3240939>.
- [36] X. Zhang, W. Pan, R. Scattolini, S. Yu, X. Xu, Robust tube-based model predictive control with Koopman operators, *Automatica* 137 (2022) 110114, <https://doi.org/10.1016/j.automatica.2021.110114>.
- [37] Z. Chen, X. Chen, J. Liu, L. Cen, W. Gui, Learning model predictive control of nonlinear systems with time-varying parameters using Koopman operator, *Appl. Math. Comput.* 470 (2024) 128577, <https://doi.org/10.1016/j.amc.2024.128577>.
- [38] J. Fu, L. Chen, D. Zhang, J. Zhang, X. Shao, Disturbance observer-based prescribed performance predictive control for spacecraft on-orbit inspection, *J. Guid. Control Dyn.* 45 (10) (2022) 1873–1889, <https://doi.org/10.2514/1.G006406>.
- [39] X. Zhu, C. Ding, C. Ren, T. Zhang, L. Jia, L. Wu, Implementation of a robust data-driven control approach for an omni-directional mobile manipulator based on Koopman operator, *Meas. Control* 55 (9–10) (2022) 1143–1154, <https://doi.org/10.1177/00202940221094843>.
- [40] C. Ren, H. Jiang, C. Li, W. Sun, S. Ma, Koopman-operator-based robust data-driven control for wheeled mobile robots, *IEEE/ASME Trans. Mechatron.* 28 (1) (2022) 461–472, <https://doi.org/10.1109/TMECH.2022.3203518>.
- [41] R. Jin, P. Rocco, X. Chen, Y. Geng, LPV-based offline model predictive control for free-floating space robots, *IEEE Trans. Aerosp. Electron. Syst.* 57 (6) (2021) 3896–3904, <https://doi.org/10.1109/TAES.2021.3083408>.
- [42] S.R. Nekoo, Output- and state-dependent Riccati equation: an output feedback controller design, *Aerosp. Sci. Technol.* 126 (2022) 107649, <https://doi.org/10.1016/j.ast.2022.107649>.
- [43] Y. Liu, X. Wang, Z. Tang, N. Qi, Probabilistic ensemble neural network model for long-term dynamic behavior prediction of free-floating space manipulators, *Aerosp. Sci. Technol.* 119 (2021) 107138, <https://doi.org/10.1016/j.ast.2021.107138>.
- [44] W. Lu, G. Huang, J. Wang, S. Hong, Spatio-temporal dynamic mode decomposition in a shear layer flow, *Aerosp. Sci. Technol.* 91 (2019) 263–271, <https://doi.org/10.1016/j.ast.2019.05.035>.
- [45] N.F. Giannelis, O. Levinski, G.A. Vio, Origins of atypical shock buffet motions on a supercritical aerofoil, *Aerosp. Sci. Technol.* 107 (2020) 106304, <https://doi.org/10.1016/j.ast.2020.106304>.
- [46] B. Lusch, J.N. Kutz, S.L. Brunton, Deep learning for universal linear embeddings of nonlinear dynamics, *Nat. Commun.* 9 (1) (2018) 4950, <https://doi.org/10.1038/s41467-018-07210-0>.
- [47] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [48] D. Yarotsky, Error bounds for approximations with deep relu networks, *Neural Netw.* 94 (2017) 103–114, <https://doi.org/10.1016/j.neunet.2017.07.002>.
- [49] E. Yeung, S. Kundu, N. Hodas, Learning deep neural network representations for Koopman operators of nonlinear dynamical systems, in: *2019 American Control Conference (ACC)*, IEEE, 2019, pp. 4832–4839.
- [50] M. Li, L. Jiang, Deep learning nonlinear multiscale dynamic problems using Koopman operator, *J. Comput. Phys.* 446 (2021) 110660, <https://doi.org/10.1016/j.jcp.2021.110660>.
- [51] X. Wang, Y. Kang, Y. Cao, Deep Koopman operator based model predictive control for nonlinear robotics systems, in: *2021 6th IEEE International Conference on Advanced Robotics and Mechatronics (ICARM)*, IEEE, 2021, pp. 931–936.
- [52] D.P. Kingma, J. Ba, Adam: a method for stochastic optimization, preprint, arXiv: 1412.6980, 2014.
- [53] L. Song, J. Wang, J. Xu, A data-efficient reinforcement learning method based on local Koopman operators, in: *2021 20th IEEE International Conference on Machine Learning and Applications (ICMLA)*, IEEE, 2021, pp. 515–520.
- [54] S. Shao, M. Chen, Y. Zhang, Adaptive discrete-time flight control using disturbance observer and neural networks, *IEEE Trans. Neural Netw. Learn. Syst.* 30 (12) (2019) 3708–3721, <https://doi.org/10.1109/TNNLS.2019.2893643>.
- [55] J. Sarangapani, *Neural Network Control of Nonlinear Discrete-Time Systems*, CRC Press, 2018.
- [56] G. Pin, D.M. Raimondo, L. Magni, T. Parisini, Robust model predictive control of nonlinear systems with bounded and state-dependent uncertainties, *IEEE Trans. Autom. Control* 54 (7) (2009) 1681–1687, <https://doi.org/10.1109/TAC.2009.2020641>.
- [57] D.L. Marruedo, T. Alamo, E.F. Camacho, Input-to-State Stable MPC for Constrained Discrete-Time Nonlinear Systems with Bounded Additive Uncertainties, *Proceedings of the 41st IEEE Conference on Decision and Control*, vol. 4, IEEE, 2002, pp. 4619–4624.
- [58] J. Virgili-Llop, et al., SPART: an open-source modeling and control toolkit for mobile-base robotic multibody systems with kinematic tree topologies, <https://github.com/NPS-SRL/SPART>.
- [59] D. Cserecsik, P. Kádár, Performance analysis of Matlab solvers in the case of a quadratic programming generation scheduling optimization problem, *World Academy of Science, Engineering and Technology, International Journal of Energy and Power Engineering* 4 (2017) 8–12, <https://api.semanticscholar.org/CorpusID:64441972>.
- [60] H. Alzer, M.K. Kwong, On Young's inequality, *J. Math. Anal. Appl.* 469 (2) (2019) 480–492, <https://doi.org/10.1016/j.jmaa.2018.06.061>.